

Naor-Yung Transform for IND-CCA Probing Security with Lattice Instantiations

Katharina Boudgoust[✉], Laurent Imbert[✉], Loïc Masure[✉], and Laz Panard

CNRS, Univ. Montpellier, LIRMM, France
firstname.lastname@lirmm.fr

Abstract. In this work, we propose novel security notions for encryption schemes that simulate an adversary in the black-box model equipped with additional side-channel power. More concretely, the adversary is allowed to probe values of the secret-key sensitive algorithms, i.e. key generation and decryption. We then prove a generalization of the well-known Naor-Yung (NY) transform, generically lifting IND-CPA secure encryption schemes to IND-CCA ones in this new probing context. Moreover, we instantiate the resulting framework from lattices, constructing **Rutile**, a masking-friendly IND-CPA encryption scheme inspired by Kyber, and then **Topaz** its IND-CCA secure extension. In our proposal, the masking-unfriendly parts of Kyber, namely the central binomial distributions and the FO-transform, are replaced by masking-friendly counterparts (sum of uniforms and the aforementioned NY-transform).

Keywords: Probing Security · Lattices · Naor-Yung-Transform · M-LWE

1 Introduction

In general any channel which can carry information from a secure area to the outside should be studied as a potential risk. [...] Vulnerable algorithms, protocols, and systems need to be revised to incorporate measures to resist timing cryptanalysis and related attacks.

It is with these words that P. Kocher concluded his seminal article “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS and Other Systems” 30 years ago [Koc96]. At the time, the reaction of the cryptographic community was as significant as Kocher’s new threat was unexpected. Constant-time implementation soon became the norm. And with the advent of differential side-channel attacks [KJJ99], advanced protections such as algebraic masking techniques were proposed. In classical public-key cryptography both types of countermeasures are relatively easy to implement. For example, cryptosystems based on the discrete logarithm problem in finite groups – such as the multiplicative subgroup of $\mathbb{Z}/p\mathbb{Z}$ or the group of rational points on an elliptic curve – can be protected by adding a random multiple of the group order to any secret value.

It was around the same time that P. W. Shor proved that the problems underlying group-based cryptography (as well as RSA) can be solved in polynomial time on a quantum computer [Sho94]. After this discovery, cryptographers began to explore quantum-safe cryptographic primitives based on error correcting codes (McEliece [McE78]), euclidean lattices (NTRU [HPS98]) or multivariate systems (HFE [Pat96]). But active, organized research didn't take off until the mid-2000s, when governments and industry began discussing migration to post-quantum cryptography. In 2016, NIST launched the PQC standardization project. In the original call for proposals and the status reports published after each round, resistance to side-channel attacks was suggested but never considered as a primary objective. For example, after the second round [AASA⁺20], one could read:

NIST hopes to see more and better data for performance in the third round. This performance data will hopefully include implementations that protect against side-channel attacks, such as timing attacks, power monitoring attacks, fault attacks, etc.

During the third round, the community addressed this request very thoroughly and proposed a large number of side-channel attacks on post-quantum schemes together with countermeasures (see [HPA21] for a survey). This phase revealed that some of the building blocks of post-quantum cryptography, relevant for security against an adversary in a standard (black-box) threat model, could be very harmful in leaky threat models [UXT⁺22], hence emphasizing how concerning it may be to consider the leakage resistance of cryptographic primitives as an afterthought, rather than jointly with black-box security.

Despite, NIST launched an additional call for proposals for digital signatures in 2022 [oST22], disregarding resistance against side-channel analysis as one of the main selection criteria.¹ Yet, among the candidates for the first round, del Pino, Katsumata, Prest, and Rossi introduced Raccoon [dPKPR24], a digital signature scheme aimed at being an alternative to Dilithium, based on the same hardness assumptions on structured lattices [LDK⁺22]. Raccoon is the first example where both provable security (EUF-CMA), *and* side-channel security are addressed simultaneously at the design level, for a reasonable cost. The main claim of the Raccoon design is to be *masking-friendly*, meaning that the specifications allow the implementation to be easy to protect with the *masking* counter-measure [ISW03]. More precisely, it consists in encoding any secret or sensitive variable according to an additive secret sharing, and to adapt the subsequent computations accordingly. In a nutshell, a masking scheme is expected to verify the *probing* security, an extension of the privacy property of secret sharing: any set of intermediate computations whose size is strictly smaller than the number of shares must be independent of any secret value. This, in turn, implies a resilience to more realistic classes of leakages [DDF14], with a security level increasing exponentially fast with the number of shares.

¹ The term “side-channel” occurs only once in the NIST status report on the first round [ABC⁺24].

Yet, this leakage resilience must be balanced with the performance overhead incurred by masking. Whereas masking linear operations is mild thanks to the homomorphism of additive secret sharing, masking non-linear operations is usually more expensive, occurring at least a quadratic overhead in terms of runtime, memory, and randomness consumption [ISW03]. In this regard, the great advantage of the masking-friendly approach of Raccoon is to deliberately weaken the masking of non-linear operations in order to avoid this expensive quadratic overhead. Nevertheless, the authors proved essentially that “whatever attack a probing adversary can carry out when knowing the leakage, she can also run (with similar success probability) by just having black-box access” [DDF14] to a variant of Raccoon, relying on weaker-still-strong hardness assumptions. As a result, the overall overhead remains reasonable, *i.e.*, quasi-linear with the number of shares, as empirically verified by the Raccoon designers [dPEK⁺23, Sec. 3.3]. This contrasts with masked implementations of Dilithium, one of the two lattice-based signatures selected by NIST for standardisation. The complexity of such masked implementations scales at least quadratically in the number of shares and has prohibitively large constants [CGTZ23]. Hence, the novel approach of designing masking-friendly primitives seems a promising solution to circumvent the tough challenge of protecting already-standardised primitives.

Contributions: In this work, we extend the line of work initiated by Raccoon [dPKPR24] on masking-friendly signatures to masking-friendly encryption schemes. We introduce Rutile, a masking friendly IND-CPA public-key encryption scheme and its IND-CCA extension Topaz, both based on lattice problems.² We extend the usual security notions for encryption schemes by providing the adversary with additional side-channel power.

Our new encryption scheme comes from the same design template as IND-CPA Kyber [BDK⁺18], which can itself be seen as an optimized version of the original Regev encryption scheme [Reg05]. Over the last decade, multiple optimizations lead to smaller ciphertexts and faster computations, like moving to structured lattices, compressing ciphertexts, working over an NTT-compatible ring and deriving large uniform elements from small seeds. The final Kyber scheme is then obtained by lifting the IND-CPA secure scheme with the Fujisaki-Okamoto (FO) transform to an IND-CCA secure KEM. The end result is the fruit of more than ten years of research in lattice-based cryptography. It has recently been selected by NIST for standardization under the name ML-KEM [oST24].

In this work, our approach was to rethink different design choices of Kyber with side-channel resistance in mind. We identified two major masking-unfriendly aspects of Kyber, namely the central binomial distributions which we replaced by sums of uniforms à la Raccoon, and the FO-transform [HLM⁺23, HHMS25] for which we opted for an alternative transformation due to Naor and Young [NY90] in its variant form proposed by Lindell [Lin06]. Unlike the FO-transform which requires decrypting a potentially forged ciphertext before

² Following the trend of CRYSTALS-Kyber, we name our schemes after two gemstones.

verifying its actual legitimacy, the NY-transform only requires to do the decryption *after* publicly verifying a zero-knowledge proof. Because the FO-transform requires the manipulation of the secret key, masking its different steps is mandatory [BGR⁺21, BC22]. Despite numerous efforts from the community, the NTT multiplications [PPM17, HSST23], the compression/decompression [WBD24] of the message and the final comparison have proved very difficult to safeguard against side-channel attacks. Conversely, the main idea of the NY-transform is to force maliciously generated ciphertexts to be rejected *before* decryption. This is achieved by encrypting the message twice with different public keys and by invoking a non-interactive zero-knowledge (NIZK) proof system on the encryptions. If verified, the proof guarantees that the two provided ciphertexts are indeed valid ciphertexts of the same message with respect of the two public keys. In this case, any of the two encryptions of the message can be safely decrypted.

The NY-transform is particularly well suited to the $(d-1)$ -probing model because the verification of the proof is independent of the secret key. Therefore the only critical operation that needs to be masked is the decryption of the underlying $(d-1)$ -probing IND-CPA scheme.

The first part of our work, presented in Section 3 and Appendix B, applies to generic public-key encryption schemes. The main contributions herein are:

- First, we propose novel security notions for encryption schemes and key encapsulation mechanisms, which we call $(d-1)$ -probing IND-CPA/IND-CCA. These new notions allow the adversary to probe up to $d - 1$ intermediate values of secret-key sensitive algorithms.
- We then prove a generalization of the well-known NY-transform to these new security notions: if equipped with a fitting NIZK, any $(d-1)$ -probing IND-CPA secure encryption scheme can be lifted to a $(d-1)$ -probing IND-CCA secure one. To account for leakage coming from the decryption queries, we require the decryption procedure to fulfill some non-interference property.³
- Once our $(d-1)$ -probing IND-CCA encryption scheme built, we can further transform it into an $(d-1)$ -probing IND-CCA key encapsulation mechanism.
- As an additional contribution, we extend the NY-transform to non-perfectly correct encryption schemes. This is particularly relevant for lattice-based instantiations, which usually allow for small decryption failures. As the NIZK only proves set membership, the adversary can provide valid NIZK's for biased encryption randomness. To account for this, we introduce the notion of *NY-compatible* encryption schemes.

The second part of our work, presented in Sections 4-7, instantiates the above framework from lattices. Our contributions can be summarized as:

- We first propose Rutile and prove its $(d-1)$ -probing IND-CPA security based on the hardness of M-LWE. To be more precise, we first reduce its security to a new variant of M-LWE which we term *probing* M-LWE and then show

³ Non-interference is a simulation-based security notion in leakage-resilience cryptography, which we formally define in Section 2.

that the latter is equivalent to standard M-LWE with secret/errors of reduced variance. Moreover we show that Rutile is NY-compatible and that its decryption algorithm fulfills the required non-interference property.

- We then select a lattice-based NIZK [LNP22] from the literature and show how to use it for the NY-transform of Rutile. Its security is based on M-SIS and extended M-LWE.
- We put everything together and propose **Topaz**, the first masking-friendly IND-CCA secure encryption scheme from lattice assumptions. Using the Lattice Estimator [APS15], we suggest a set of parameters that reach a 128-bit security level.
- Finally, we give a comparison with Kyber and argue both better side-channel security and masking performance heuristically.

In terms of ease of implementation, our scheme inherits the same advantages as Raccoon, not only from an implementation efficiency perspective, but also in terms of security analysis. On the one hand, our error distributions are based on sums of uniform distributions over $\{-2^{u-1}, \dots, 2^{u-1} - 1\}$, which makes its implementation straightforward across a wide variety of platforms. Besides, unlike Kyber, our scheme does not require masked implementations of symmetric components so that the number of distinct masking operations to implement is reduced. On the other hand, the masking-friendly part of our scheme shares common building blocks (called *gadgets* in the masking terminology), with those of Raccoon, in particular the addition of small noisy elements. It turns out that these gadgets have recently been analyzed in the so-called *random* probing model [BRR25], a more realistic variant of the probing model. This suggests that the security of our scheme nicely extends to this more realistic threat scenario. There only remains the masked 1-bit compression, which is admittedly the leakiest part (and hence the most vulnerable). Nevertheless, this is not believed to be the most harmful attack vector in Kyber [UXT⁺22], which suggests a competitive advantage in terms of leakage resilience in favor of our approach.

Future Directions: We present some interesting future directions.

Decreasing the Efficiency Gap. We see our proposed encryption schemes and the resulting concrete parameters as a first base line, demonstrating the efficiency loss when replacing the FO-transform by the masking-friendly NY-transform. As we haven't used the full potential of optimizing our schemes, we would be thrilled to see the public key and ciphertext sizes decreasing in the future.

Towards a full Masking Friendliness. Making the noise generation masking-friendly, like in the Raccoon signature scheme, and replacing the FO-transform by a verification proof independent of the secret is a great step towards a masking-friendly public-key encryption scheme. But our construction is not fully masking-friendly yet, as the runtime and memory overhead remains quadratic with the number of shares d . More precisely, like in Kyber, we still rely on a non-linear compression operation, with complexity $\mathcal{O}(d^2 \cdot \log \log q)$ [CGMZ23,

[CGMZ21, App. D]. In this respect, finding a masking-friendly alternative – *i.e.*, with $\mathcal{O}(d \cdot \log d)$ overhead – to this bottleneck represents a promising track for improvement in future works. Another direction would be to additionally mask the encryption algorithm (which requires masking the LNP NIZK), hence extending **Topaz** probing security to the message as well.

Probing Security, beyond Encryption. We extended the probing-leakage resilience paradigm of Raccoon to public-key encryption, and subsequently to key encapsulation mechanism. A potential track for future works could be to extend this paradigm to more advanced primitives. As an example, the Raccoon signature scheme has been extended to threshold signatures [DKM⁺24]. Given the parallel with our probing-secure encryption scheme, a natural extension could be to explore the potential of our approach for threshold encryption schemes.

Related Works: A notable related work is the lattice-based IND-CCA encryption scheme Polka [HLM⁺23]. In our work as well as in [HLM⁺23], leakage resistance was taken into account at the design level. However, Polka and **Topaz** differ in the methodology to achieve leakage resistance. Polka’s approach is to look at existing side-channel attacks and to give heuristic arguments (based on novel non-standard lattice problems) how their choices reduce the attack surface. This attack surface has been recently refined in [PPS26]. Our approach, in contrast, is to construct a scheme proven secure in the probing model with better masking-friendliness. We think it would be interesting to combine the insights of both approaches in future works.

Besides Raccoon [dPKPR24], other masking-friendly and presumably post-quantum signature schemes have been proposed in the literature, namely lattice-based Plover [EEN⁺24], and masking-friendly variants of the Threshold-Computation-in-the-Head Framework [FRW25].

In our model, the IND-CCA adversary can probe values during the executions of the key generation and decryption algorithms. We can thus see it as *computation leakage*. A different model is to provide the adversary with a separate leakage oracle on the secret key, for arbitrary (but bounded) leakage functions, as studied in [AGV09, NS09]. This can be categorized as *memory leakage* (as for instance done in this survey on leakage-resilient cryptography [KR19]). In an analogue manner to our work, it was shown in [NS09] that one can use the Naor-Yung transform to lift IND-CPA to IND-CCA security in the memory leakage model. Note that the impact of computation leakage on the NY-transform is more important than the impact of memory leakage, as in the computation leakage case the adversary is allowed to probe the additional decryption oracle.

2 Preliminaries

For $d \in \mathbb{N}^*$, we note $[d] = \{1, \dots, d\}$. For $x \in \mathbb{R}$, we note $\lfloor x \rfloor$ the nearest integer to x , half rounded up. Let $i, j \in (\mathbb{N}^*)^2$, we note $\mathbf{0}_{i,j}$ the $i \times j$ zero matrix. For $n = 2^\ell > 1$, we denote by $\mathcal{R}$ the ring $\mathbb{Z}[X]/\langle X^n + 1 \rangle$, and for modulus

q , we denote by $\mathbb{Z}_q$ the ring $\mathbb{Z}/q\mathbb{Z}$, and by $\mathcal{R}_q$ the ring $\mathbb{Z}_q[X]/\langle X^n + 1 \rangle$. For $f = \sum_{i=0}^{n-1} f_i X^i \in \mathcal{R}$, we note the l_∞ -norm $\|f\|_\infty = \max_i(|f_i|)$ and the l_2 -norm $\|f\|_2 = \sqrt{\sum_i f_i^2}$. For $k \in \mathbb{N}^*$ and $\mathbf{f} = (f_1, \dots, f_k) \in \mathcal{R}^k$, we note the l_∞ -norm $\|\mathbf{f}\|_\infty = \max_i(\|f_i\|_\infty)$ and the l_2 -norm $\|\mathbf{f}\|_2 = \sqrt{\sum_i \|f_i\|_2^2}$. Both for f and $\mathbf{f}$ we sometimes write $\|\cdot\|$ instead of $\|\cdot\|_2$. When writing the norm of an element in $\mathcal{R}_q$, we implicitly take the centered representation modulo q , interpret it as element in $\mathcal{R}$ and then compute the norm.

Lemma 1. *Let $k \in \mathbb{N}^*$ and $\mathbf{a}, \mathbf{b} \in (\mathcal{R}^k)^2$, $\|\mathbf{a}^t \mathbf{b}\|_\infty \leq \|\mathbf{a}\|_2 \cdot \|\mathbf{b}\|_2$.*

We provide a proof in Appendix A.1 for completeness.

For $u \in \mathbb{N}^*$, $\text{Unif}(u)$ denotes the uniform distribution over the set $\{-2^{u-1}, \dots, 2^{u-1} - 1\}$. By interpreting every element in $\mathcal{R}$ as a polynomial of degree less than n with coefficients in $\mathbb{Z}$, for $k \in \mathbb{N}^*$, we denote by $\text{Unif}(\mathcal{R}, k, u)$ the distribution over $\mathcal{R}^k$ which samples vectors of k polynomials whose n coefficients are independently sampled from $\text{Unif}(u)$. Put differently, $\text{Unif}(\mathcal{R}, k, u) = \text{Unif}(u)^{n \cdot k}$. By $\text{NSUnif}(u, l)$ we denote the (non-zero-centered) distribution where l elements are sampled independently and identically from $\text{Unif}(u)$ and summed together. For a random variable $X \sim \text{NSUnif}(u, l)$, one can check that $X + l/2$ is sub-Gaussian for $\sigma^2 = 2^{2u} l/6$. By $\text{SUnif}(u, l) = \text{NSUnif}(u, l) + l/2$ we denote the sum of uniform distribution which is now zero-centered, while preserving the variance. We extend it in the same way over $\mathcal{R}^k$, denoting $\text{NSUnif}(\mathcal{R}, k, u, l)$ the sum of l copies of $\text{Unif}(\mathcal{R}, k, u)$. Similarly, we define $\text{SUnif}(\mathcal{R}, k, u, l) = \text{NSUnif}(\mathcal{R}, k, u, l) + l/2$, where by abuse of notation, the latter means the vector of polynomials whose coefficients are all $l/2$. Note that $\text{SUnif}(\mathcal{R}, k, u, l) = \text{SUnif}(u, l)^{n \cdot k}$.

The tail bound from [LPR13, Lem. 2.2] applies to $\mathbf{x} \leftarrow \text{SUnif}(\mathcal{R}, k, u, l)$, thus:

$$\Pr \left[\|\mathbf{x}\|_2 > 2^u \sqrt{3lnk} \right] \leq \exp(-2nk). \quad (1)$$

For two distributions χ and ψ , we note $\chi \stackrel{i.d.}{=} \psi$ when the two distributions are identically distributed. This notation naturally extends to random variables that are identically distributed, as well as random functions. We write $x \leftarrow \chi$ when x is a value sampled according to χ and $x \leftarrow S$ when x is sampled uniformly at random in the finite set S .

Let f a random function, we note $x \leftarrow f(\cdot)$ when x is a random output of f . To explicit the random coins R used in f , we note $x := f(\cdot; R)$, where $:=$ highlights the fact that x is non-probabilistic.

2.1 The (Probing) M-LWE Problem

Let us recall in Def. 1 the Module Learning With Errors (M-LWE) problem, as introduced in [BGV12, LS15]. In this paper, we are interested in M-LWE instances where both the secret and the noise are sampled as sums of small uniforms. With a suitable parameter selection, for instance derived using the Lattice

Estimator [APS15], this is considered a standard hardness assumption in lattice-based cryptography. Putting additional parameter restrictions, its hardness can be derived from worst-case problems on module lattices [BJRW23].

In Definition 2, we introduce a new variant of M-LWE, where up to $d - 1$ summands of the secret and noise are provided to the adversary. We call this the $(d-1)$ -probing M-LWE problem and show that it is equivalent to standard M-LWE where secret and error follow a (smaller) sum of uniform distribution. For simplicity, we stick to power-of-two cyclotomic rings and square matrices in this section, but everything generalizes to arbitrary cyclotomic rings and rectangular matrices as well. Whereas in this section, we only use the word "probes" in an informal manner, we give a formal definition of the term "probes" in the context of probing security in Section 2.3.

Definition 1 (M-LWE). Let $n \in \mathbb{N}^*$ be a power of two and $q \in \mathbb{N}^*$ be a modulus, defining the ring $\mathcal{R}_q$. Let $l \in \mathbb{N}^*$ be the number of small uniforms to sum, $u \in \mathbb{N}^*$ their size parameter and $k \in \mathbb{N}^*$ the dimensions of the matrix over $\mathcal{R}_q$. The experiment of $\text{M-LWE}_{\mathcal{R}_q, k, l, u}$ is defined in Figure 1a. For an adversary $\mathcal{A}$ trying to solve the M-LWE problem, the respective advantage is defined as

$$\text{Adv}(\mathcal{A}) = \left| \Pr[\text{M-LWE}_{\mathcal{R}_q, k, l, u}(\mathcal{A}) = 1] - \frac{1}{2} \right|.$$

Upon this, we build the so-called $(d-1)$ -probing M-LWE problem, which is used in the security proof of Rutile in Section 7.1.

Definition 2 ($(d-1)$ -probing M-LWE). Let l, u, k, n and q be integers as in the previous definition, setting $\mathcal{R}_q$, and let $d - 1 \in \mathbb{N}$ with $l > d - 1$, the number of probes allowed to the attacker. The $(d-1)$ -probing $\text{M-LWE}_{\mathcal{R}_q, d, k, l, u}$ experiment is defined in Figure 1b. For an adversary $\mathcal{A}$, the respective advantage is defined as

$$\text{Adv}(\mathcal{A}) = \left| \Pr[(d-1)\text{-probing M-LWE}_{\mathcal{R}_q, d, k, l, u}(\mathcal{A}) = 1] - \frac{1}{2} \right|.$$

If $d = 1$, we recover standard M-LWE from Definition 1. Proposition 1 shows that $(d-1)$ -probing M-LWE is equivalent to M-LWE with a reduced number of summed small uniforms. The proof only uses the fact that the secret and error distributions of M-LWE are sums of *independently and identically distributed* samples of the same distribution. In our case, this distribution is the uniform distribution, as it is suitable in the additive masking setting, but it can easily be exchanged with other distributions if needed. The proof essentially uses the linearity of M-LWE and can be found in Appendix A.2.

Proposition 1. Let $n \in \mathbb{N}^*$ be a power of two and $q \in \mathbb{N}^*$ be a modulus, defining the ring $\mathcal{R}_q$. Let $l \in \mathbb{N}^*$ be the number of small uniforms to sum, $u \in \mathbb{N}^*$ their size parameter, $k \in \mathbb{N}^*$ the dimension of matrix and $d \in \mathbb{N}^*$ the number of "probes" plus one allowed to the attacker, with $l > d - 1$. Then,

$$\text{M-LWE}_{\mathcal{R}_q, k, (l-d+1), u} \Leftrightarrow (d-1)\text{-probing M-LWE}_{\mathcal{R}_q, d, k, l, u}.$$

M-LWE $_{\mathcal{R}_q, k, l, u}(\mathcal{A})$	$(d-1)$ -probing M-LWE $_{\mathcal{R}_q, d, k, l, u}(\mathcal{A})$
1 : $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times k}$	1 : $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times k}$
2 : $b \leftarrow \{0, 1\}$	2 : $\{1, \dots, l\} \not\supseteq \overline{\mathcal{P}} \leftarrow \mathcal{A}(\mathbf{A})$
3 : if $b = 0$:	3 : if $ \overline{\mathcal{P}} \geq d$
4 : for $i \in \{1, \dots, l\}$	4 : return $\perp$
5 : $\rho_i^{(s)}, \rho_i^{(e)} \leftarrow (\text{Unif}(\mathcal{R}, k, u))^2$	5 : for $i \in \{1, \dots, l\}$
6 : $\mathbf{s} = \sum_{i=1}^l \rho_i^{(s)}, \mathbf{e} = \sum_{i=1}^l \rho_i^{(e)}$	6 : $\rho_i^{(s)}, \rho_i^{(e)} \leftarrow (\text{Unif}(\mathcal{R}, k, u))^2$
7 : $\mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$	7 : $\mathcal{L} = (\rho_i^{(s)}, \rho_i^{(e)})_{i \in \overline{\mathcal{P}}}$
8 : else :	8 : $b \leftarrow \{0, 1\}$
9 : $\mathbf{b} \leftarrow \mathcal{R}_q^k$	9 : if $b = 0$:
10 : $b' \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{b})$	10 : $\mathbf{s} = \sum_{i=1}^l \rho_i^{(s)}, \mathbf{e} = \sum_{i=1}^l \rho_i^{(e)}$
11 : return $b = b'$	11 : $\mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$
	12 : else :
	13 : $\mathbf{b} \leftarrow \mathcal{R}_q^k$
	14 : $b' \leftarrow \mathcal{A}(\mathbf{b}, \mathcal{L})$
	15 : return $b = b'$
(a) The M-LWE experiment	(b) The $(d-1)$ -probing M-LWE experiment

Fig. 1: M-LWE experiments in the classical (black-box) security model and in the (grey-box) $(d-1)$ -probing model

More precisely, there exists an adversary $\mathcal{A}$ against M-LWE $_{\mathcal{R}_q, k, (l-d+1), u}$ with advantage Adv if and only if there exists an adversary $\mathcal{B}$ against the problem $(d-1)$ -probing M-LWE $_{\mathcal{R}_q, d, k, l, u}$ with the same advantage Adv .

2.2 Cryptographic Primitives

We recall the definitions of public-key encryption schemes and non-interactive zero-knowledge proof systems.

Definition 3 (PKE). A public-key encryption scheme PKE over message space $\mathcal{M}$ is defined by the following PPT algorithms (KeyGen, Enc, Dec), parameterized by the security parameter κ , where:

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$: on input the security parameter κ , the key generation algorithm outputs a public key pk and a secret key sk .
- $\text{ct} \leftarrow \text{Enc}(\text{pk}, m)$: on input a public key pk and a plaintext $m \in \mathcal{M}$, the encryption algorithm outputs a ciphertext ct .
- $m' \leftarrow \text{Dec}(\text{pk}, \text{sk}, \text{ct})$: given a public key pk , a secret key sk and a ciphertext ct , the decryption algorithm outputs a plaintext $m' \in \mathcal{M}$ or the symbol $\perp$.

To make the random coins used in Enc explicit, we write $\text{ct} := \text{Enc}(\text{pk}, m; R)$, with R the random coins sampled from the randomness distribution RandDistr .

Definition 4 (Correctness). Let $\delta_{\text{fail}}(\kappa)$ be a function parameterized by the security level κ taking real values in $[0, 1]$. A $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ over message space $\mathcal{M}$ with randomness distribution RandDistr is called $\delta_{\text{fail}}(\kappa)$ -correct, if for all $\kappa \in \mathbb{N}^*$, for all $m \in \mathcal{M}$ it holds

$$\Pr_{\substack{R \leftarrow \text{RandDistr} \\ (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)}} [\text{Dec}(\text{pk}, \text{sk}, \text{Enc}(\text{pk}, m; R)) \neq m] \leq \delta_{\text{fail}}(\kappa).$$

We further use the primitive of one-time simulation-sound NIZK's. We assume that the prover and verifier both have access to the same random oracle during the execution of the protocol.

Definition 5 (NIZK). Let H be a hash function modeled as a random oracle. For an algorithm A , we denote by A^H the fact that the algorithm has access to the random oracle H . A non-interactive zero-knowledge proof system (NIZK) for a relation R consists of a pair of probabilistic algorithms $\Pi = (P, V)$, where

- $P^H(x, w) \rightarrow \pi$: The prover algorithm takes as input a statement witness pair $(x, w) \in R$, and outputs a proof π .
- $V^H(x, \pi) \rightarrow b \in \{0, 1\}$: The verifier algorithm takes as input a statement x and proof π , and outputs accept ($b = 1$) or reject ($b = 0$).

We denote by $L = \{x : \exists w, (x, w) \in R\}$ the language induced by R .

Definition 6 (Completeness). Let $\epsilon_{\text{complete}}(\kappa)$ be a function parameterized by the security parameter κ taking real values in $[0, 1]$. A NIZK $\Pi = (P, V)$ for the relation R and with associated random oracle H is called $\epsilon_{\text{complete}}(\kappa)$ -complete if for all $k \in \mathbb{N}^*$, all $(x, w) \in R$ it holds

$$\Pr [V^H(x, P^H(x, w)) \neq 1] \leq \epsilon_{\text{complete}}(\kappa),$$

where the probability is taken over the randomness of P, V and the random oracle H .

Definition 7 (Zero-Knowledge). Let $\epsilon_{\text{ZK}}(\kappa)$ be a function parameterized by the security parameter κ taking real values in $[0, 1]$. A NIZK $\Pi = (P, V)$ for the relation R and with associated random oracle H is called $\epsilon_{\text{ZK}}(\kappa)$ -zero-knowledge if there exists a PPT simulator $\text{Sim} = (\text{Sim}_H, \text{Sim}_\pi)$ such that for all $\kappa \in \mathbb{N}$ and for all PPT adversaries $\mathcal{A}$ it holds that

$$\left| \Pr [\text{Expt}_{\mathcal{A}}^{\text{ZK-Real}}(1^\kappa) = 1] - \Pr [\text{Expt}_{\mathcal{A}, \text{Sim}}^{\text{ZK-Sim}}(1^\kappa) = 1] \right| \leq \epsilon_{\text{ZK}}(\kappa),$$

where the probability is taken over the random coins of the adversary, the random oracle H and the experiments defined in Figure 2a.

Definition 8 (One-Time Simulation Soundness). Let $\epsilon_{\text{OTS}}(\kappa)$ be a function parameterized by the security parameter κ taking real values in $[0, 1]$ and let $\text{Sim} = (\text{Sim}_H, \text{Sim}_\pi)$ be a zero-knowledge simulator. A NIZK $\Pi = (P, V)$ for

the relation R and with associated random oracle H is called $\epsilon_{\text{OTSS}}(\kappa)$ -one-time simulation sound if for every $\kappa \in \mathbb{N}$, for all PPT adversaries $\mathcal{A}$, it holds that

$$\Pr \left[\text{Expt}_{\mathcal{A}, \text{Sim}}^{\text{OTSS}}(1^\kappa) = 1 \right] \leq \epsilon_{\text{OTSS}}(\kappa),$$

where the probability is taken over the random coins of the adversary and the experiments defined in Figure 2b.

We denote by OTSS-NIZK a NIZK that is $\epsilon_{\text{complete}}(\kappa)$ -complete, $\epsilon_{\text{ZK}}(\kappa)$ -zero-knowledge and $\epsilon_{\text{OTSS}}(\kappa)$ -one-time simulation sound, where all three functions are negligible in κ .

$\text{Expt}_{\mathcal{A}}^{\text{ZK-Real}}(1^\kappa)$	$\text{Expt}_{\mathcal{A}, \text{Sim}}^{\text{ZK-Sim}}(1^\kappa)$	$\text{Expt}_{\mathcal{A}, \text{Sim}}^{\text{OTSS}}(1^\kappa)$
1 : $(x, w) \leftarrow \mathcal{A}^H(1^\kappa)$	1 : $(x, w) \leftarrow \mathcal{A}^{\text{Sim}_H}(1^\kappa)$	1 : $x \leftarrow \mathcal{A}^{\text{Sim}_H}(1^\kappa)$
2 : if $(x, w) \notin R$	2 : if $(x, w) \notin R$	2 : $\pi \leftarrow \text{Sim}_\pi(x)$
3 : return $\perp$	3 : return $\perp$	3 : $(x', \pi') \leftarrow \mathcal{A}^{\text{Sim}_H}(\pi)$
4 : $\pi \leftarrow P(x, w)$	4 : $\pi \leftarrow \text{Sim}_\pi(x)$	4 : $b = V(x', \pi') \wedge (x' \notin L)$
5 : $b \leftarrow \mathcal{A}(\pi)$	5 : $b \leftarrow \mathcal{A}(\pi)$	5 : $\wedge ((x', \pi') \neq (x, \pi))$
6 : return b	6 : return b	6 : return b
(a) Zero-knowledge		(b) One-time simulation soundness

Fig. 2: Security experiments of non-interactive proof systems.

2.3 Probing Theory

Wires, Values, Programs and Probes. We recall that an *arithmetic circuit* is a direct acyclic graph that contains addition and multiplication nodes (or gates) as well as input and output nodes. We assume that the arithmetic nodes $(+, \times)$ have input degree 2 and output degree 1. Input nodes have input degree zero and output nodes have output degree zero. These nodes are connected by edges or *wires* that we note with overhead letters (e.g. $\bar{x}$). For an execution of the algorithm, the *value* carried by the wire labeled $\bar{x}$ is simply denoted x . These notations naturally extend to a set of wires (e.g. $\bar{\mathcal{X}}$) and its corresponding tuple of values $\mathcal{X}$.

We use the notation $\llbracket x \rrbracket = (x_i)_{i \in [d]}$ to denote a tuple of d values, which implicitly defines the value $x = \sum_{i=1}^d x_i$. Put differently, the encoding $\llbracket x \rrbracket$ means that the (secret) value x is shared as d additive shares.

We shall need the following privacy property of additive secret sharing: Let $I \subsetneq [d]$ be any index subset of size at most $d - 1$ and $\llbracket x \rrbracket = (x_i)_{i \in [d]}$ be the masking of an arbitrary value x over some finite ring $\mathcal{R}$. Then, the distribution $((x_i)_{i \in I}, x)$ is identical to the distribution $((y_i)_{i \in I}, x)$, where $y_i \leftarrow \mathcal{R}$.

We denote by **KeyUpdate** a function which takes as input a d -sharing $\llbracket K \rrbracket = (K_i)_{i \in [d]}$ of some value K and outputs another d -sharing of the same value $\llbracket K \rrbracket = (K'_i)_{i \in [d]}$ such that for every two sets of indices $I_1, I_2 \subsetneq [d]$, we have that $(K_i)_{i \in I_1}$ and $(K'_i)_{i \in I_2}$ are independent random variables. This implies that having access to at most $d - 1$ values of $(K_i)_{i \in [d]}$ does not leak any information about $(K'_i)_{i \in [d]}$ nor about K .

Likewise, for an arithmetic circuit which takes as input a masked value $\llbracket x \rrbracket$, we denote $\llbracket \vec{x} \rrbracket = (\vec{x}_i)_{i \in [d]}$ the d -tuple of input wires carrying the d shares of $\llbracket x \rrbracket$.

Definition 9 (Randomized Arithmetic Circuit). *A randomized arithmetic circuit is an arithmetic circuit which also contains nodes with input degree zero and output degree one that output random values.*

Definition 10 (Program). *We define a program Prog as a randomized arithmetic circuit, and denote by $\overline{W}_{\text{Prog}}$ (or $\overline{W}$ when clear from context) its set of wires. The set of input wires is partitioned into $\overline{\mathcal{X}}_{\text{masked}}$ whose wires carry sharings of secret values, and $\overline{\mathcal{X}}_{\text{unmasked}}$ whose wires carry unmasked secret values. We have $\overline{\mathcal{X}}_{\text{Prog}} = \overline{\mathcal{X}}_{\text{masked}} \cup \overline{\mathcal{X}}_{\text{unmasked}} \subset \overline{W}_{\text{Prog}}$. Similarly, we denote by $\overline{\mathcal{Y}} = (\overline{\mathcal{Y}}_{\text{public}}, \overline{\mathcal{Y}}_{\text{secret}})$ the mutually disjoint sets of respectively public and secret (masked or unmasked) output wires. We define $\mathcal{X}$ and $\mathcal{Y}$ the sets of input and output values such that $\mathcal{Y} \leftarrow \text{Prog}(\mathcal{X})$.*

We note that in the above definition of a program we abstract public inputs as implicit parameters of the program.

We now introduce the notion of probes on a program. On a high level, the notion of probes formalizes the idea that an attacker retrieves a number of intermediate values during the execution of the program, thus allowing for a potential leakage of secret values. In the so-called probing model, a *probe* is the label of an algorithm's wire targeted by the attacker, and a *leakage* is the value carried by this wire that is thus provided to the attacker.

Definition 11 (Probes). *Let Prog be a program as in Def. 10, with set of input wires $\overline{\mathcal{X}} = (\overline{\mathcal{X}}_{\text{masked}}, \overline{\mathcal{X}}_{\text{unmasked}})$. A set of probes is a subset $\overline{\mathcal{P}}$ of Prog 's wires. Given a set of probes $\overline{\mathcal{P}}$ and $\mathcal{X}$ the values carried by the program's input wires, we note $\text{Prog}^{\mathcal{E}}(\mathcal{X}, \overline{\mathcal{P}})$ the joint distribution*

$$(\mathcal{Y}, \mathcal{L}) \leftarrow \text{Prog}^{\mathcal{E}}(\mathcal{X}, \overline{\mathcal{P}}),$$

where $\mathcal{Y} = \mathcal{Y}_{\text{public}} \cup \mathcal{Y}_{\text{secret}}$ denotes the public and secret outputs of $\text{Prog}(\mathcal{X})$ for some internal random coins, and where the leakage $\mathcal{L}$ is the set of values carried by the wires in $\overline{\mathcal{P}}$ for these same random coins.

Simulatability and Non-Interference Properties. We now introduce the notions of (strong) non-interference without unmasked inputs [BBD⁺16] and with unmasked inputs [dPKPR24].

Definition 12 (NI/Nlu property, [BBD⁺16, dPKPR24]). *Let $d \in \mathbb{N}^*$ and Prog be a program with input wires $\overline{\mathcal{X}} = (\overline{\mathcal{X}}_{\text{masked}}, \overline{\mathcal{X}}_{\text{unmasked}})$. We say that*

Prog is (d-1)-Non-Interfering with Unshared inputs ((d-1)-Nlu for short) if and only if there exist two PPT simulators **SimProb** and **SimLeak** such that for any input values $\mathcal{X}$ and any set of probes $\overline{\mathcal{P}}$ with $|\overline{\mathcal{P}}| < d$, we have that:

- **SimProb**($\overline{\mathcal{P}}$) outputs $\overline{\mathcal{X}}' = (\overline{\mathcal{X}}'_{\text{masked}}, \overline{\mathcal{X}}'_{\text{unmasked}})$ where:
 - $\overline{\mathcal{X}}'_{\text{masked}} \subset \overline{\mathcal{X}}_{\text{masked}}$ and $\overline{\mathcal{X}}'_{\text{unmasked}} \subset \overline{\mathcal{X}}_{\text{unmasked}}$
 - for any tuple of wires of a masked variable $\llbracket x \rrbracket$, $|\llbracket x \rrbracket \cap \overline{\mathcal{X}}'_{\text{masked}}| \leq |\overline{\mathcal{P}}|$
 - $|\overline{\mathcal{X}}'_{\text{unmasked}}| \leq |\overline{\mathcal{P}}|$
- **SimLeak**($\mathcal{X}', \overline{\mathcal{P}}$) output's distribution is identical to that of $\mathcal{L}$, where $\mathcal{X}' \subset \mathcal{X}$ is the set of values carried by the wires in $\overline{\mathcal{X}}'$ and $(\mathcal{Y}, \mathcal{L}) \leftarrow \text{Prog}^{\mathcal{L}}(\mathcal{X}, \overline{\mathcal{P}})$

When **Prog** has no unmasked secret input (i.e. $\overline{\mathcal{X}}_{\text{unmasked}} = \emptyset$), we simply call it (d-1)-Non-Interfering ((d-1)-NI for short).

In general, $\overline{\mathcal{X}}'$ is a strict subset of $\overline{\mathcal{X}}$ as masking will be done on level d , but the simulator is only allowed to simulate using less than d wires.

Definition 13 (sNI/sNlu property, [BBD⁺16, dPKPR24]). Let $d \in \mathbb{N}^*$ and **Prog** be a program with input wires $\overline{\mathcal{X}} = (\overline{\mathcal{X}}_{\text{masked}}, \overline{\mathcal{X}}_{\text{unmasked}})$ and output wires $\overline{\mathcal{Y}}$. We say that **Prog** is (d-1)-Strongly Non-Interfering with Unshared inputs ((d-1)-sNlu for short) if and only if it meets the same criteria than to be Nlu, except that for any set of probes $\overline{\mathcal{P}}$, upper bound on **SimProb** outputs is $|\overline{\mathcal{P}} \setminus \overline{\mathcal{Y}}|$ instead of $|\overline{\mathcal{P}}|$. When $\overline{\mathcal{X}}_{\text{unmasked}} = \emptyset$, we simply call **Prog** (d-1)-Strongly Non-Interfering ((d-1)-sNI for short).

We now introduce non interference with public outputs properties where the public output of the program is set and not random as in definitions above. This notion is introduced in the random probing model in [BRR25, Def. 5]. We reformulate this definition and extend it to the d -probing model.

Definition 14 (Nlo/Nluo property, [BRR25]). Let $d \in \mathbb{N}^*$ and **Prog** be a program with input wires $\overline{\mathcal{X}} = (\overline{\mathcal{X}}_{\text{masked}}, \overline{\mathcal{X}}_{\text{unmasked}})$ and public output wires $\overline{\mathcal{Y}}_{\text{public}}$. We say that **Prog** is (d-1)-Non-Interfering with Unshared inputs and public Outputs ((d-1)-Nluo for short) if and only if there exists two PPT simulators **SimProb** and **SimLeak** such that for any input values $\mathcal{X}$ and public output values $\mathcal{Y}_{\text{public}}$ for which $\Pr(\overline{\mathcal{X}} = \mathcal{X}, \overline{\mathcal{Y}}_{\text{public}} = \mathcal{Y}_{\text{public}}) \neq 0$, and any set of probes $\overline{\mathcal{P}}$ where $|\overline{\mathcal{P}}| < d$, **SimProb** and **SimLeak** meets the same criteria than in Def. 12 when **SimLeak** is given access to $\mathcal{Y}_{\text{public}}$.

When $\overline{\mathcal{X}}_{\text{unmasked}} = \emptyset$, we simply call **Prog** (d-1)-Non-Interfering with public Outputs ((d-1)-Nlo for short).

We now introduce a useful result for composition of a Nlu program with a Nlo program whose public output is deterministic in the Nlu gadget input.

Proposition 2 (Nlu+Nlo = Nluo). Let $d \in \mathbb{N}^*$, **Prog**₁ a (d-1)-Nlu program with a single d -shared masked input variable $\overline{\mathcal{X}}_1 = (\llbracket x_1 \rrbracket, \overline{\mathcal{X}}_{\text{unmasked}})$, no public output

and a single d -shared masked variable as secret output $\overline{\mathcal{Y}}_1 = (\emptyset, \llbracket \overline{y} \rrbracket)$ and Prog_2 a $(d-1)$ -Nlo program with a single d -shared masked input variable $\overline{\mathcal{X}}_2 = (\llbracket \overline{x}_2 \rrbracket, \emptyset)$ and output $\overline{\mathcal{Y}}_2 = (\overline{\mathcal{Y}}_{\text{public}}, \emptyset)$. If there exist a deterministic function $f : \mathcal{R}^{|\overline{\mathcal{X}}_1|} \rightarrow \mathcal{R}^{|\overline{\mathcal{Y}}_2|}$ such that for every input values $\mathcal{X}$ carried by $\overline{\mathcal{X}}$ we have $\mathcal{Y}_2 = f(\mathcal{X})$ with $\mathcal{Y}_2$ the output values carried by $\overline{\mathcal{Y}}_2$ wires, then Prog is $(d-1)$ -Nluo where, for every input $\mathcal{X} = (\llbracket x \rrbracket, \mathcal{X}_{\text{unmasked}})$,

$$\text{Prog}(\llbracket x \rrbracket, \mathcal{X}_{\text{unmasked}}) := \text{Prog}_2(\text{Prog}_1(\llbracket x \rrbracket, \mathcal{X}_{\text{unmasked}}))$$

Proof Sketch. Without loss of generality we assume there are d_2 probes on Prog_2 and d_1 probes on Prog_1 with $d_1 + d_2 < d$. The overall idea of our proof is that the d_2 probes on Prog_2 may be simulated with values carried by up to d_1 wires of $\overline{\mathcal{X}}_2$, which can in turn be simulated along with the d_1 probes on Prog_1 — i.e. simulating less than d values — by Prog_1 's Nlu simulator. A full proof is provided in Appendix A.3.

Simulating with public output only The following result shows that the leakage of a $(d-1)$ -Nlo program can be simulated by a simulator which only takes the set of probes and public output values as input. The proof essentially leverages the privacy property of the additive secret sharing used for the masked values. For simplicity, we show our result in the case of one single masked value, but it easily generalizes with some additional notational overhead to multiple masked input values. We do not claim this to be a novel observation, but we were not able to find a result in the literature which exactly proves what we need. Hence, we provide a proof for completeness in Appendix A.4.

Proposition 3. *For $d \in \mathbb{N}^*$, let Prog be a $(d-1)$ -Nlo program with no unmasked input (i.e. $\overline{\mathcal{X}}_{\text{unmasked}} = \emptyset$), only one masked input $\overline{\mathcal{X}}_{\text{masked}} = \llbracket \overline{x} \rrbracket$ with d wires and such that the public output $\mathcal{Y}_{\text{public}}$ of Prog is uniquely determined by its public input and the value for which $\llbracket \overline{x} \rrbracket$ carries the d -sharing (and thus not by its concrete sharing). There exists a simulator Sim such that for every set of probes $\overline{\mathcal{P}}$ s.t. $|\overline{\mathcal{P}}| < d$, for any $x \in \mathcal{R}$ and any privacy-preserving additive sharing $\llbracket x \rrbracket$ of x we have that*

$$\mathcal{L} \stackrel{i.d.}{=} \text{Sim}(\mathcal{Y}_{\text{public}}, \overline{\mathcal{P}}),$$

where $(\mathcal{Y}_{\text{public}}, \mathcal{Y}_{\text{secret}}, \mathcal{L}) \leftarrow \text{Prog}^{\mathcal{S}}(\text{KeyUpdate}(\llbracket x \rrbracket), \overline{\mathcal{P}})$.

3 Semantic Probing Security for PKE

In this section, we propose new security notions for public-key encryption (PKE) schemes. They can be seen as a combination of (black-box) cryptographic semantic and (grey-box) probing security notions, which are usually considered separately. More concretely, for $d \in \mathbb{N}$, we define the notion of $(d-1)$ -probing IND-CPA in Section 3.1 and $(d-1)$ -probing IND-CCA in Section 3.2. We further demonstrate in Section 3.3 that the well-known Naor-Young transform [NY90], lifting IND-CPA to IND-CCA, applies even if augmented with the probing security

when querying (up to a polynomial number of times) the decryption oracle on ciphertext ct , the adversary can additionally ask for probes $\overline{\mathcal{P}}$. As an answer, they receive the decrypted message m together with the leakage $\mathcal{L}$. As done in the context of masked signatures [BBE⁺18, dPKPR24], we assume the existence of an algorithm called **KeyUpdate** (as formalized in Section 2.3) which refreshes the sharing of the secret key between two decryption queries. As explained in Section 3.1, we do not allow probes on **Enc**. We further assume that the attacker cannot probe this algorithm. The purpose of this algorithm is to make sure that the adversary cannot learn more than $d - 1$ shares of the secret key across different decryption queries.

$(d-1)$ -probing IND-CCA for PKE, with $b \in \{0, 1\}$	$\text{DEC}_{\llbracket \text{sk} \rrbracket, \text{ct}^*}(\overline{\mathcal{P}}, \text{ct})$
1 : $\overline{\mathcal{P}}_{\text{KG}} \leftarrow \mathcal{A}(1^\kappa)$	1 : if $(\overline{\mathcal{P}} \geq d) \vee (\text{ct} = \text{ct}^*) :$
2 : $(\text{pk}, \llbracket \text{sk} \rrbracket, \mathcal{L}_{\text{KG}}) \leftarrow \text{KeyGen}^\mathcal{L}(1^\kappa, \overline{\mathcal{P}}_{\text{KG}})$	2 : return $\perp$
3 : $(m_0, m_1) \leftarrow \mathcal{A}^{\text{DEC}_{\llbracket \text{sk} \rrbracket, \emptyset}}(\text{pk}, \mathcal{L}_{\text{KG}})$	3 : $\llbracket \text{sk} \rrbracket \leftarrow \text{KeyUpdate}(\llbracket \text{sk} \rrbracket)$
4 : $\text{ct}^* \leftarrow \text{Enc}(\text{pk}, m_b)$	4 : return $\text{Dec}^\mathcal{L}(\text{ct}, \overline{\mathcal{P}})$
5 : $b^* \leftarrow \mathcal{A}^{\text{DEC}_{\llbracket \text{sk} \rrbracket, \text{ct}^*}}(\text{ct}^*)$	
6 : return b^*	

$$\text{WIN}_{\text{PKE}}^{\text{IND-CCA}}(b) = (b = b^*) \wedge (|\overline{\mathcal{P}}_{\text{KG}}| \leq d - 1)$$

Fig. 4: $(d-1)$ -probing IND-CCA for PKE security game

Definition 16. A PKE scheme is called $(d-1)$ -probing IND-CCA secure if it holds that for all security parameters $\kappa \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$ in the game described in Figure 4 we have:

$$\text{Adv}_{\mathcal{A}}^{\text{dCCA}}(\kappa) := |\Pr[\text{WIN}^{\text{IND-CCA}}(0)] - \Pr[\text{WIN}^{\text{IND-CCA}}(1)]| \leq \text{negl}(\kappa),$$

We note that in the security game, we only require probing security for the **KeyGen** and the **Dec** procedures, leaving the **Enc** procedure as it is. We also note that once again our definition coincides with the usual "no-probing" definition when $d = 1$.

Confirming intuition, we provide in Appendix B with Lemma 16 a formal proof that the $(d-1)$ -probing IND-CCA security notion gets stronger for larger allowed number of probes. In particular, Corollary 2 implies that probing IND-CCA security is at least as strong as standard IND-CCA security.

3.3 Lifting Probing IND-CPA to Probing IND-CCA

We conclude the section by proving our main theoretical result, showing that a well-known transformation lifting IND-CPA to IND-CCA security generalizes to

the notions allowing for probes. Concretely, we use a variant of the Naor-Young transform [NY90] as presented in [Lin06] (but in the random oracle model), which we call NY-transform in short. It uses the power of publicly verifiable NIZK's to guarantee the well-formedness of ciphertexts. As a side contribution, we formally prove the Naor-Young transform for non-perfectly correct schemes, for which we introduce the notion of *NY-compatible* encryption schemes.

NY-Compatible PKE Let Σ be a public-key encryption scheme with message space $\mathcal{M}$ and randomness distribution RandDistr , and let RandSpace be a set. Within the NY-transform, we switch from RandDistr to RandSpace , as the NIZK only proves set membership, but nothing about the underlying distribution. This is particularly relevant in lattice-based instantiations, where the underlying scheme is not perfectly correct. We define the relation R_Σ to be used in the NY-transform as follows:

$$R_\Sigma := \left\{ (x, w) \left| \begin{array}{l} x = (\text{ct}_1, \text{ct}_2, \text{pk}_1, \text{pk}_2) \\ w = (m, R_1, R_2) \in \mathcal{M} \times \text{RandSpace}^2 \\ \text{ct}_i := \text{Enc}(\text{pk}_i, m; R_i) \quad i \in \{1, 2\} \end{array} \right. \right\} \quad (2)$$

We denote by $L_\Sigma = \{x : \exists w, (x, w) \in R_\Sigma\}$ the language induced by R_Σ .

We now introduce the notion of *NY-compatible* encryption schemes. This property ensures two things. First, it bounds the probability that honestly sampled encryption randomness $R \leftarrow \text{RandDistr}$ falls outside of RandSpace . Second, it bounds the probability that there exist message-randomness tuples such that for honestly generated keys, ciphertexts produced using the message-randomness tuple lead to conflicting decryptions. This is the key to leverage the NY-transform to non-perfectly correct schemes, as a valid NIZK proof not only guarantees that the ciphertexts encrypt the same message, but also that they decrypt to the same message for honest key pairs.

Definition 17. Let $\Sigma = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a PKE scheme with message space $\mathcal{M}$, randomness distribution RandDistr and let RandSpace be a set. We say that Σ is *NY-compatible* for RandSpace with parameter $\hat{\delta}_{\text{fail}}(\kappa)$ if for all security parameters κ it holds $\max(P_1, P_2) \leq \hat{\delta}_{\text{fail}}(\kappa)$, where

$$P_1 := \Pr_{R \leftarrow \text{RandDistr}} [R \notin \text{RandSpace}],$$

$$P_2 := \Pr \left[\exists (m, R_1, R_2) \in \mathcal{M} \times \text{RandSpace}^2 \left| \begin{array}{l} \text{Dec}(\text{sk}_1, \text{Enc}(\text{pk}_1, m; R_1)) \\ \neq \text{Dec}(\text{sk}_2, \text{Enc}(\text{pk}_2, m; R_2)) \end{array} \right. \right],$$

with the latter probability taken over $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(1^\kappa)$ for $i \in \{1, 2\}$.

Note that in the above definition we implicitly require that one can efficiently verify if randomness R lies in RandSpace or not.

Theorem 1. We denote by κ the security parameter.

Let $\Sigma = (\text{KeyGen}_{\text{CPA}}, \text{Enc}_{\text{CPA}}, \text{Dec}_{\text{CPA}})$ be a $(d-1)$ -probing IND-CPA secure PKE scheme with message space $\mathcal{M}$ and randomness distribution RandDistr and space RandSpace such that Dec_{CPA} is deterministic and $(d-1)$ -Nlo. Moreover, let Σ be NY-compatible for RandSpace with parameter $\hat{\delta}_{\text{fail}}(\kappa)$.

Further, let H be a hash function modeled as a random oracle and let $\Pi = (\text{P}^H, \text{V}^H)$ be a OTSS-NIZK for the relation R_Σ as defined in Equation 2.

Then, the PKE scheme $\Sigma' = (\text{KeyGen}_{\text{CCA}}, \text{Enc}_{\text{CCA}}, \text{Dec}_{\text{CCA}})$ described in Algorithms 2-3 with the same message space $\mathcal{M}$ is $(d-1)$ -probing IND-CCA secure.

More concretely, assuming that Π is $\epsilon_{\text{complete}}(\kappa)$ -complete and that Σ is $\delta_{\text{fail}}(\kappa)$ -correct, then the constructed Π' is $\delta'_{\text{fail}}(\kappa)$ -correct with $\delta'_{\text{fail}}(\kappa) \leq \delta_{\text{fail}}(\kappa) + \epsilon_{\text{complete}}(\kappa)$.

Moreover, assuming that Π is $\epsilon_{\text{OTSS}}(\kappa)$ -OTSS and $\epsilon_{\text{ZK}}(\kappa)$ -ZK, every PPT adversary $\mathcal{A}$ against Σ' with advantage $\text{Adv}_{\mathcal{A}}^{\text{dCCA}}(\kappa)$ defines adversary $\mathcal{B}$ against Σ with advantage $\text{Adv}_{\mathcal{B}}^{\text{dCPA}}(\kappa)$, where

$$\text{Adv}_{\mathcal{A}}^{\text{dCCA}}(\kappa) \leq \epsilon_{\text{OTSS}}(\kappa) + \hat{\delta}_{\text{fail}}(\kappa) + 2(\epsilon_{\text{ZK}}(\kappa) + \text{Adv}_{\mathcal{B}}^{\text{dCPA}}(\kappa)).$$

Algorithm 1 Enc_{CCA}

Input: public key pk | message m

Output: ciphertext ct

```

pk = (pk1, pk2)
flag = false
while flag = false do
  R1  $\leftarrow$  RandDistr
  R2  $\leftarrow$  RandDistr
  if R1, R2  $\in$  RandSpace then
    flag = true
  end if
end while
ct1  $\leftarrow$  EncCPA(pk1, m; R1)
ct2  $\leftarrow$  EncCPA(pk2, m; R2)
x = (pk1, pk2, ct1, ct2)
w = (m, R1, R2)
 $\pi \leftarrow \text{P}^H(x, w)$ 
ct := (ct1, ct2,  $\pi$ )

```

Algorithm 2 $\text{KeyGen}_{\text{CCA}}$

Input: security parameter κ

Output: public key pk | secret key sk

```

(pk1,  $\llbracket \text{sk}_1 \rrbracket$ )  $\leftarrow$  KeyGenCPA(1 $\kappa$ )
(pk2,  $\llbracket \text{sk}_2 \rrbracket$ )  $\leftarrow$  KeyGenCPA(1 $\kappa$ )
pk := (pk1, pk2)
sk :=  $\llbracket \text{sk}_1 \rrbracket$ 

```

Algorithm 3 Dec_{CCA}

Input: secret key sk | public key pk | ciphertext ct

Output: message m

```

pk = (pk1, pk2)
sk =  $\llbracket \text{sk}_1 \rrbracket$ 
ct = (ct1, ct2,  $\pi$ )
x := (pk1, pk2, ct1, ct2)
if VH(x,  $\pi$ ) = 1 then
  m := DecCPA(pk1,  $\llbracket \text{sk}_1 \rrbracket$ , ct1)
else
  m :=  $\perp$ 
end if

```

The high level overview of the proof is depicted in Figure 5. The most important difference to the original NY-transform [Lin06] are the additional first and last steps (Lemma 2+7), which handle probing security related issues. Moreover, we replace the common reference string model by the random oracle model. To better follow the proof idea, we denote the hybrids by $(b_1, b_2)_{\Delta}^*$, where

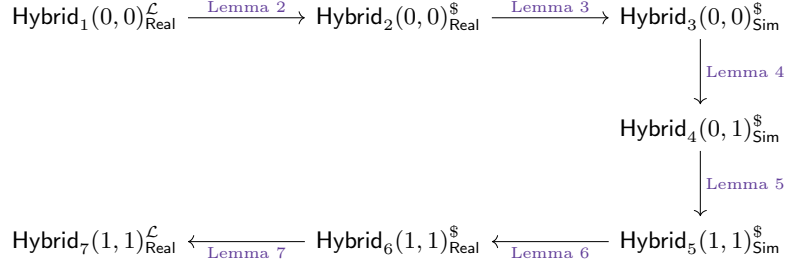


Fig. 5: Reduction Overview of NY with Probing Transformation.

$b_1, b_2 \in \{0, 1\}$ denote which messages are encrypted in the challenge ciphertext, $\star \in \{\mathcal{L}, \$\}$ denotes whether the leakage on the decryption procedure is real or simulated and $\Delta \in \{\text{Real}, \text{Sim}\}$ designs whether random oracle and proof of the NIZK are real or simulated. The sub proofs which essentially follow the original NY-transform are deferred to Appendix B.3.

Proof. We prove the security reduction via a sequence of hybrids. Proof of correctness can be found in App. B.3

Hybrid₁ (0, 0)_{Real}^ℒ: The starting hybrid corresponds to the original experiment of $(d-1)$ -probing IND-CCA detailed in Figure 4 with bit $b = 0$.

Hybrid₂ (0, 0)_{Real}^ℒ: In the second hybrid, we change the decryption oracle, modifying how the answers $(m_i, \mathcal{L}_i)$ are produced. Everything else stays the same. Let $\text{ct}_i = (\text{ct}_{1i}, \text{ct}_{2i}, \pi_i)$ and $\overline{\mathcal{P}}_i$ be the input to the decryption oracle. As specified in Algorithm 3, Dec_{CCA} does essentially two things: verifying the NIZK prove π and invoking Dec_{CPA} on input $(\text{pk}_1, \llbracket \text{sk}_1 \rrbracket, \text{ct}_{1i})$. As the verification of the NIZK is a public procedure, we can assume without loss of generality that the probes $\overline{\mathcal{P}}_i$ are only on the algorithm Dec_{CPA} . Let Sim_{Dec} be the simulator defined in Proposition 3 for the $(d-1)$ -Nlo program Dec_{CPA} . Instead of running the leaky decryption $(m_i, \mathcal{L}_i) \leftarrow \text{Dec}_{\text{CPA}}^{\mathcal{L}}(\text{pk}_1, \llbracket \text{sk}_1 \rrbracket, \text{ct}_{1i}, \overline{\mathcal{P}}_i)$, we first run the normal decryption $m_i \leftarrow \text{Dec}_{\text{CPA}}(\text{pk}_1, \llbracket \text{sk}_1 \rrbracket, \text{ct}_{1i})$ without any probes and leakage. Then, we call Sim_{Dec} on the input $(\text{ct}_{1i}, \text{pk}_1, m_i, \overline{\mathcal{P}}_i)$ to obtain the simulated output $\mathcal{L}_i$. Note that now, the leakage $\mathcal{L}_i$ does not depend on the secret key sk_1 anymore.

Hybrid₃ (0, 0)_{Sim}^ℒ: The third hybrid is the same as Hybrid₂, except how the random oracle is modeled and how the prove π during challenge ciphertext generation is constructed. Let $\text{Sim}_{\text{NIZK}} = (\text{Sim}_H, \text{Sim}_\pi)$ be the simulator from the OTSS-NIZK definition. Instead of a truly random oracle H , we let the simulator Sim_H simulate H . Moreover, we replace the real proof π^* contained in ct^* by a simulated one issued by Sim_π .

Hybrid₄ (0, 1)_{Sim}^ℒ: The fourth hybrid is the same as Hybrid₃, except how the challenge ciphertext is produced. Instead of encrypting twice the same message m_0 under the two different public keys pk_1, pk_2 , we keep ct_1 as an encryption of m_0 under pk_1 . However, we now produce ct_2 as the encryption of m_1 under pk_2 .

Hybrid₅ $(1, 1)_{\text{Sim}}^{\$}$: The fifth hybrid is the same as **Hybrid₄**, except we modify again how the challenge ciphertext is produced. Instead of setting ct_1 as the encryption of m_0 under pk_1 , we let it be the encryption of m_1 , still under the public key pk_1 . We thus have ct_1 and ct_2 be encryptions of the same message m_1 .

Hybrid₆ $(1, 1)_{\text{Real}}^{\$}$: The sixth hybrid is the same as **Hybrid₅**, except we modify how the random oracle H is modeled and how the proof π during challenge ciphertext generation is constructed — the change from **Hybrid₂** to **Hybrid₃** in reverse order. Instead of letting the simulator Sim_{NIZK} simulate H , we model it as a truly random oracle. Moreover, we replace the simulated proof π^* contained in ct^* by a real one issued by P .

Hybrid₇ $(1, 1)_{\text{Real}}^{\mathcal{L}}$: The seventh hybrid is the same as **Hybrid₆**, except we modify how the leakage for the decryption oracle is constructed — the change from **Hybrid₁** to **Hybrid₂** in the reverse order. Instead of using Sim_{Dec} on the input $(\text{ct}_i, \text{pk}, \overline{\mathcal{P}}_i)$ to get the leakage $\mathcal{L}_i$ and of running the non-leaky usual decryption $\text{Dec}(\text{pk}, \llbracket \text{sk} \rrbracket, \text{ct}_i)$ to get the message m_i , we run the leaky decryption $\text{Dec}^{\mathcal{L}}(\text{pk}, \llbracket \text{sk} \rrbracket, \text{ct}_i, \overline{\mathcal{P}}_i)$ to get both the message m_i and the leakage $\mathcal{L}_i$. Note that this corresponds to security game of $(d-1)$ -probing IND-CCA with $b = 1$.

We now prove the advantages for distinguishing between consecutive hybrids.

Lemma 2 (Hybrid₁ to Hybrid₂). *Assume the decryption algorithm Dec_{CPA} of Σ is deterministic (d-1)-Nlo. Then, the advantage in distinguishing **Hybrid₁** from **Hybrid₂** is zero.*

Proof. Recall that the security notion of $(d-1)$ -probing IND-CCA invokes before answering to a decryption query the **KeyUpdate** algorithm which refreshes the sharing of the secret key $\text{sk} = \text{sk}_1$. By the properties of **KeyUpdate** (and the fact that the model does not allow the adversary to probe this algorithm), we know that the (at most $d - 1$) probes on the sharing $\llbracket \text{sk} \rrbracket$ during a new query to the decryption oracle are independent to any information the adversary might have obtained from previous probes on $\llbracket \text{sk} \rrbracket$. Hence, it is enough to argue for a single decryption query.

We now argue that Dec_{CPA} fulfills the conditions of Proposition 3. It has no unmasked secret input, only one d -masked secret input $\llbracket \text{sk}_1 \rrbracket$ and its public input is given by $(\text{pk}_1, \text{ct}_{1i})$. Moreover, its public output m_i is uniquely defined by $(\text{pk}_1, \text{ct}_{1i}, \text{sk}_1)$ (not depending on the exact sharing of sk_1 , and no secret output). Thus, all conditions of Proposition 3 are met, guaranteeing the existence of a simulator Sim_{Dec} which perfectly simulates leakage $\mathcal{L}_i$ without needing the masked input $\llbracket \text{sk}_1 \rrbracket$. ■

Proofs for Lemma 3+4 can be found in Appendix B.3.

Lemma 3 (Hybrid₂ to Hybrid₃). *Let Π be $\epsilon_{\text{ZK}}(\kappa)$ -zero-knowledge. Then, for any PPT adversary $\mathcal{A}$, the advantage in distinguishing **Hybrid₂** from **Hybrid₃** is upper bounded by $\epsilon_{\text{ZK}}(\kappa)$.*

Lemma 4 (Hybrid₃ to Hybrid₄). *Let Σ be $(d-1)$ -probing IND-CPA secure. That is, any PPT adversary $\mathcal{B}$ against $(d-1)$ -probing IND-CPA of Σ has advantage at*

most $\text{Adv}_{\mathcal{B}}^{\text{dCPA}}(\kappa)$. Then, for any PPT adversary $\mathcal{A}$, the advantage in distinguishing Hybrid_3 from Hybrid_4 is upper bounded by $\text{Adv}_{\mathcal{B}}^{\text{dCPA}}(\kappa)$.

Lemma 5 (Hybrid₄ to Hybrid₅). *Let (Σ, Π) be $\widehat{\delta}_{\text{fail}}(\kappa)$ -NY-compatible and let Σ be $(d-1)$ -probing IND-CPA secure. The latter is, any PPT adversary $\mathcal{B}$ against $(d-1)$ -probing IND-CPA of Σ has advantage at most $\text{Adv}_{\mathcal{B}}^{\text{dCPA}}(\kappa)$. Moreover, let Π be $\epsilon_{\text{OTSS}}(\kappa)$ -one-time-simulation-sound. For any PPT adversary $\mathcal{A}$, the advantage in distinguishing Hybrid_4 from Hybrid_5 is upper bounded $\text{Adv}_{\mathcal{B}}^{\text{dCPA}}(\kappa) + \widehat{\delta}_{\text{fail}}(\kappa) + \epsilon_{\text{OTSS}}(\kappa)$.*

Proof. For simplicity, we omit the dependency on the security parameter κ . Let (Σ, Π) be $\widehat{\delta}_{\text{fail}}$ -NY-compatible and Π be ϵ_{OTSS} -OTSS. Further, let $\mathcal{A}$ be a PPT adversary distinguishing between Hybrid_4 and Hybrid_5 with advantage at least Adv , then we build a PPT adversary $\mathcal{B}$ that breaks the $(d-1)$ -probing IND-CPA security of Σ with advantage $\text{Adv}_{\mathcal{B}}^{\text{dCPA}}$ at least $\text{Adv} - \widehat{\delta}_{\text{fail}} - \epsilon_{\text{OTSS}}$.

We begin by describing the reduction. We proceed similarly to the proof of Lemma 4, except we now embed the $(d-1)$ -probing IND-CPA challenge in pk_1 and ct_1 instead of pk_2 and ct_2 . First $\mathcal{B}$ receives from $\mathcal{A}$ as input a set of probes $\overline{\mathcal{P}}_{KG} = (\overline{\mathcal{P}}_{KG1}, \overline{\mathcal{P}}_{KG2})$ on the key generation procedure. The key pair $(\text{pk}_2, \text{sk}_2)$ and leakage $\mathcal{L}_{KG2}$ are computed by $\mathcal{B}$ who runs $\text{KeyGen}_{\text{CPA}}^{\mathcal{E}}$, while pk_1 and $\mathcal{L}_{KG1}$ are input to them after they output $\overline{\mathcal{P}}_{KG1}$ as their query. Then $\mathcal{B}$ sends the public key $\text{pk} = (\text{pk}_1, \text{pk}_2)$, and the leakage $\mathcal{L}_{KG} = (\mathcal{L}_{KG1}, \mathcal{L}_{KG2})$ to $\mathcal{A}$. When $\mathcal{B}$ receives m_0 and m_1 from $\mathcal{A}$, they have the ciphertext ct_1 be input to them after forwarding m_0 and m_1 as output. Then $\mathcal{B}$ computes ct_2 by calling Enc_{CPA} on m_1 and pk_2 and they compute π by calling the simulator Sim_{π} on $x = (\text{pk}_1, \text{pk}_2, \text{ct}_1, \text{ct}_2)$. They then send the challenge ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2, \pi)$ to $\mathcal{A}$. When $\mathcal{B}$ is input with a decryption query, they perform decryption by first verifying the proof π , then running Dec_{CPA} on sk_2 — instead of sk_1 — and the ciphertext queried, as they do not have access to sk_1 . The leakage simulator continues to run on the same input as before. The reduction $\mathcal{B}$ outputs as their guess $b^* = 0$ if $\mathcal{A}$ guesses Hybrid_3 , and $b^* = 1$ if $\mathcal{A}$ guesses Hybrid_4 .

The proof of correct simulation of the views of $\mathcal{A}$ is exactly the same than in Lemma 4's proof, except for decryption queries. Let $\text{ct} = (\text{ct}_1, \text{ct}_2, \pi)$ be any ciphertext queried to $\mathcal{B}$. As the NIZK proof system Π is ϵ_{OTSS} -OTSS and Σ is $\widehat{\delta}_{\text{fail}}$ -NY-compatible, we have that there exist no decryption query such that ct_1 and ct_2 are not encryptions of the same message or such that $\text{Dec}_{\text{CPA}}(\text{ct}_1, \text{sk}_1) \neq \text{Dec}_{\text{CPA}}(\text{ct}_2, \text{sk}_2)$, except with probability $\epsilon_{\text{OTSS}} + \widehat{\delta}_{\text{fail}}$. We now assume that for all queries both ciphertexts are encryptions of the same message and that both decrypt to the same element. Then the distribution of the answer to decryption queries, containing both message and leakage, is perfectly simulated. Hence $\mathcal{B}$ perfectly simulates the views of $\mathcal{A}$.

As in Lemma 4's proof, we have that whenever $\mathcal{A}$ wins, if $\mathcal{B}$ perfectly simulated the views of $\mathcal{A}$ then they also win. Therefore, as $\mathcal{A}$ has advantage Adv , $\mathcal{B}$ has advantage at least $\text{Adv} - \epsilon_{\text{OTSS}} - \widehat{\delta}_{\text{fail}}$. $\blacksquare$

Lemma 6 (Hybrid₅ to Hybrid₆). *Let Π be $\epsilon_{\text{ZK}}(\kappa)$ -zero-knowledge. Then, for any PPT adversary $\mathcal{A}$, the advantage in distinguishing Hybrid₅ from Hybrid₆ is upper bounded by $\epsilon_{\text{ZK}}(\kappa)$.*

Proof. The proof of Lemma 6 is essentially the same as for Lemma 3. It suffices to replace all encryptions of m_0 by encryptions of m_1 . ■

Lemma 7 (Hybrid₆ to Hybrid₇). *Assume the decryption algorithm Dec_{CPA} of Σ is (d-1)-Nlo. Then, the advantage in distinguishing Hybrid₆ from Hybrid₇ is zero.*

Proof. The proof of Lemma 7 is essentially the same as for Lemma 2. It suffices to replace all encryptions of m_0 by encryptions of m_1 . ■

By applying all consecutive lemmas from Lemma 2 to Lemma 7, we thus have the following bound on the advantage Adv of any adversary $\mathcal{A}$ that distinguishes between Hybrid₁ and Hybrid₇ — thus concluding the proof:

$$\text{Adv} \leq \epsilon_{\text{OTSS}}(\kappa) + 2(\epsilon_{\text{ZK}}(\kappa) + \text{Adv}^{\text{dCPA}}(\kappa)) + \hat{\delta}_{\text{fail}}$$

□

4 Description of Rutile

In this section, we provide the description of Rutile, a lattice-based public-key encryption scheme. We further prove its correctness and its NY-compatibility. The probing IND-CPA security proof of Rutile is deferred to Section 7.1.

Its design is inspired both by the IND-CPA version of Kyber [BDK⁺18] and the masking-friendly signature Raccoon [dEK⁺23]. Without loss of generality, we assume that the ring degree n is lower bounded by the security parameter κ . Hence, proving that something is negligible in n implies that it is negligible in κ .

The main parameters of Rutile are the following: the power-of-two cyclotomic ring $\mathcal{R}$ of degree n modulo q , the dimension k of the public key matrix $\mathbf{A}$, the masking level d and the parameters (u, rep) for uniformly sampling short elements over $\mathcal{R}$.

Algorithm 4 $\text{KeyGen}_{\text{CPA}}$

Input: level of security κ

Output: public key $\text{pk} = (\mathbf{A} \in \mathcal{R}_q^{k \times k}, \mathbf{t} \in \mathcal{R}_q^k) \mid$ private key $\text{sk} = \llbracket \mathbf{s} \rrbracket \in (\mathcal{R}_q^k)^d$

- 1: $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times k}$
 - 2: $\llbracket \mathbf{s} \rrbracket \leftarrow \text{AddRepNoise}(\mathbf{0}_{d,1}, u, \text{rep})$ ▷ Nluc gadget for $\mathbf{s} \leftarrow \text{SUnif}(\mathcal{R}, k, u, d \cdot \text{rep})$
 - 3: $\llbracket \mathbf{e} \rrbracket \leftarrow \text{AddRepNoise}(\mathbf{0}_{d,1}, u, \text{rep})$ ▷ Nluc gadget for $\mathbf{e} \leftarrow \text{SUnif}(\mathcal{R}, k, u, d \cdot \text{rep})$
 - 4: $\llbracket \mathbf{t} \rrbracket \leftarrow \mathbf{A} \cdot \llbracket \mathbf{s} \rrbracket + \llbracket \mathbf{e} \rrbracket$
 - 5: $\mathbf{t} := \text{Decode}(\llbracket \mathbf{t} \rrbracket)$ ▷ Nlo gadget for $\mathbf{t} := \sum_i \llbracket \mathbf{t} \rrbracket_i$
 - 6: $\text{pk} := (\mathbf{A}, \mathbf{t})$
 - 7: $\text{sk} := \llbracket \mathbf{s} \rrbracket$
-

Algorithm 5 Enc_{CPA}

Input: message μ | the public key $\text{pk} = (\mathbf{A}, \mathbf{t})$

Output: ciphertext $\text{ct} = (v, \mathbf{u}) \in \mathcal{R}_q \times \mathcal{R}_q^k$

1: $\mathbf{u} \leftarrow \text{SUnif}(\mathcal{R}, k, u, d \cdot \text{rep})$

2: $\mathbf{e}_1 \leftarrow \text{SUnif}(\mathcal{R}, k, u, d \cdot \text{rep})$

3: $e_2 \leftarrow \text{SUnif}(\mathcal{R}, 1, u, d \cdot \text{rep})$

4: $\mathbf{v}^t := \mathbf{u}^t \cdot \mathbf{A} + \mathbf{e}_1^t$

5: $v := \mathbf{u}^t \cdot \mathbf{t} + e_2 + \lfloor q/2 \rfloor \mu$

6: $\text{ct} := (v, \mathbf{v})$

Algorithm 6 Dec_{CPA}

Input: ciphertext $\text{ct} = (v, \mathbf{v}) \in \mathcal{R}_q \times \mathcal{R}_q^k$ | private key $\text{sk} = \llbracket \mathbf{s} \rrbracket \in (\mathcal{R}_q^k)^d$

Output: message μ

1: $\llbracket \mu' \rrbracket \leftarrow v - \mathbf{v}^t \cdot \llbracket \mathbf{s} \rrbracket$

2: $\llbracket \mu \rrbracket \leftarrow \text{1bit-Compress}(\llbracket \mu' \rrbracket)$ $\triangleright$ NI gadget for a d -sharing of $\left\lfloor \|\mu'\|_\infty / \lfloor q/2 \rfloor \right\rfloor$

3: $\mu := \text{Decode}(\llbracket \mu \rrbracket)$ $\triangleright$ Nlo gadget for $\mu := \sum_i \llbracket \mu \rrbracket_i$

Algorithm 4 describes the key generation. It is essentially the same as the key generation in Raccoon. It generates an M-LWE instance with masking-friendly sums of uniform distributions for secret and error. Note that Kyber’s key generation uses M-LWE with centered binomials instead. The key difference is that the masking level d already appears in the definition of the secret/error distribution, as it impacts the number of summands. Concretely, we use the `AddRepNoise` gadget from [dPKPR24, Alg. 5] to sample sums of uniforms in a masked way. Actually, to be very precise, the gadget from [dPKPR24] is for non-zero centered sums of uniforms (which we called `NSUnif` in the preliminaries). It easily extends to a gadget for zero-centered sums of uniforms, simply by adding a constant off-set. The `Decode` gadget can be obtained by applying [CGMZ23, Cor. 1] to [BCRT23, Cor. 5]. More details on this can be found in Section 6.

The encryption procedure, detailed in Algorithm 5, is essentially the same as the one of the IND-CPA version of Kyber, just that for simplicity we left out the ciphertext compression at the end.⁴ Encryption randomness is again sampled from sums of uniform distributions, even though we do not require masking techniques for the encryption. Note that the message space is given by $\mathcal{M} = \{0, 1\}^n$ and the encryption randomness is coming from the distribution $\text{RandDistr} = \text{SUnif}(\mathcal{R}, 2k + 1, u, d \cdot \text{rep})$.

Algorithm 6 states the decryption procedure. It can be seen as a masked version of the decryption algorithm from the IND-CPA version of Kyber, just without the decompress coming from the compressed ciphertexts.

To simplify notations we omit the NTT representations here, but when concretely instantiating our scheme one can add the NTT without any problems.

⁴ Compressing ciphertexts would require a more sophisticated setup of the NIZK which we leave for future works.

Moreover, in practice, one usually generates the matrix $\mathbf{A}$ from a short seed $\text{seed} \leftarrow \{0, 1\}^\kappa$. A description of the 1bit-Compress gadget can be found in [BC22, Alg. 15] for 1bit-Compress.

We now state correctness of our scheme. A proof can be found in App. C.

Lemma 8. *Let $\text{PKE} = (\text{KeyGen}_{\text{CPA}}, \text{Enc}_{\text{CPA}}, \text{Dec}_{\text{CPA}})$ be the encryption scheme described in Algorithms 4-6. If $2^{2u} \cdot d \cdot \text{rep} \cdot n(6k+3) \leq \lfloor q/4 \rfloor$, then it is δ_{fail} -correct, with $\delta_{\text{fail}} = 2 \exp(-2n(2k+1))$.*

We now argue the NY-compatibility of Σ , as described in Alg. 4-6, with message space $\mathcal{M} = \{0, 1\}^n$ and encryption randomness distribution $\text{RandDistr} = \text{SUnif}(\mathcal{R}, 2k+1, u, d \cdot \text{rep})$. Set $B^{(e)} = 2^u \sqrt{3n(2k+1)d \cdot \text{rep}}$ and define $\text{RandSpace} = \{\mathbf{r} \in \mathcal{R}^{2k+1} : \|\mathbf{r}\|_2 \leq B^{(e)}\}$.

We can reformulate the encryption algorithm as follows. Let $\mathbf{r} = (\mathbf{u}, \mathbf{e}_1, e_2)$ and $\mathbf{B} = \begin{pmatrix} \mathbf{A}^t & \mathbf{I}_k & \mathbf{0}_{k,1} \\ \mathbf{t}^t & \mathbf{0}_{1,k} & 1 \end{pmatrix}$, then

$$\mathbf{c} = \mathbf{B}\mathbf{r} + \lfloor q/2 \rfloor \cdot (\mathbf{0}_{1,k}, \mu)^t \in \mathcal{R}_q^{k+1}. \quad (3)$$

With this notation, the PKE public key space is characterized by

$$\mathcal{K} = \left\{ \mathbf{B} = \begin{pmatrix} \mathbf{A}^t & \mathbf{I}_k & \mathbf{0}_{k,1} \\ \mathbf{t}^t & \mathbf{0}_{1,k} & 1 \end{pmatrix} \in \mathcal{R}_q^{k+1 \times k'} \mid \mathbf{A} \in \mathcal{R}_q^{k \times k}, \mathbf{t} \in \mathcal{R}_q^k \right\}.$$

Let R_Σ be the relation defined in Section 3.3. With our concrete instantiation of Σ at hand, we reformulate it to

$$\text{R}_\Sigma := \left\{ (x, w) \mid \begin{array}{l} x = (\mathbf{c}_1, \mathbf{c}_2, \mathbf{B}_1, \mathbf{B}_2) \in (\mathcal{R}_q^{k+1})^2 \times \mathcal{K}^2 \\ w = (\mu, \mathbf{r}_1, \mathbf{r}_2) \in \mathcal{M} \times \text{RandSpace}^2 \\ \mathbf{c}_i = \mathbf{B}_i \mathbf{r}_i + \lfloor q/2 \rfloor \cdot (\mathbf{0}_{1,k}, \mu)^t \quad i \in \{1, 2\} \end{array} \right\}.$$

We now show NY-compatibility of this scheme (see App. C for proof)

Lemma 9. *Let Σ be the encryption scheme described above with message space $\mathcal{M}$. For $B^{(e)}$ define the randomness space $\text{RandSpace} = \{\mathbf{r} \in \mathcal{R}^{2k+1} : \|\mathbf{r}\|_2 \leq B^{(e)}\}$. If $(B^{(e)})^2 \leq \lfloor q/4 \rfloor$, then Σ is NY-compatible for RandSpace with parameter $\widehat{\delta}_{\text{fail}} = 2 \exp(-2n(2k+1))$.*

Note that in our scheme the correctness parameter δ_{fail} coincides with the NY-compatibility parameter $\widehat{\delta}_{\text{fail}}$. This is not the case in general.

5 Our Instantiation of the [LNP22] NIZK

As required in Theorem 1, to lift the probing IND-CPA secure encryption scheme Rutile from Section 4 to an IND-CCA secure encryption scheme, which we call Topaz, we need a specific NIZK for proving "joint encryption". More precisely, it

needs to prove that two ciphertexts are encryptions under different public keys of a same message with overwhelming probability over the distribution of the key pair generation. To this extent we use the lattice-based NIZK proof system in the ROM from [LNP22, Sec. 5.2]. While the specific protocol we forward readers to is an interactive one, we note that Sec. 6 and App. B of [LNP22] detail its transformation to a NIZK in the ROM via the Fiat-Shamir transformation. Thus, when referring to this protocol we actually refer to its transformed NIZK version. The LNP NIZK is more expressive than what we need, thus we will only be introducing the relations of interest for this paper. For a modulus p and a statement composed of

- For $i \in [v^{(d)}]$, $\mathbf{E}_i^{(d)} \in \mathcal{R}_p^{k_i^{(d)} \times 2k' + 1}$, $\mathbf{v}_i^{(d)} \in \mathcal{R}_p^{k_i^{(d)}}$
- For $i \in [v^{(e)}]$, $\mathbf{E}_i^{(e)} \in \mathcal{R}_p^{k_i^{(e)} \times 2k' + 1}$, $\mathbf{v}_i^{(e)} \in \mathcal{R}_p^{k_i^{(e)}}$
- Bounds $(\beta_i^{(d)})_{i \in [v^{(d)}]}$, $(\beta_i^{(e)})_{i \in [v^{(e)}]}$
- Matrix $\mathbf{E}_{bin} \in \mathcal{R}_p^{k_{bin} \times 2k' + 1}$, $\mathbf{v}_{bin} \in \mathcal{R}_p^{k_{bin}}$

the LNP NIZK proves that there exists a witness vector $\mathbf{r} \in \mathcal{R}_p^{2k' + 1}$ satisfying the following relationships with overwhelming probability

$$\forall i \in [v^{(d)}], \left\| \mathbf{E}_i^{(d)} \mathbf{r} - \mathbf{v}_i^{(d)} \right\|_\infty \leq \beta_i^{(d)} \quad (4)$$

$$\forall i \in [v^{(e)}], \left\| \mathbf{E}_i^{(e)} \mathbf{r} - \mathbf{v}_i^{(e)} \right\|_2 \leq \beta_i^{(e)} \quad (5)$$

$$\mathbf{E}_{bin} \mathbf{r} - \mathbf{v}_{bin} \text{ is binary}^5 \quad (6)$$

We now explain how to use the LNP NIZK to prove joint encryption of the scheme Σ described in Section 4, where we use the formulation of encryption from Equation 3. Essentially our idea is to reproduce the instantiations of Equations 4 to 6 in the context of verifiable encryption as described in [LNP22, Sec. 6.3] twice, while forcing the use of a same message for both ciphertexts. Note that for technical reasons the modulus q used in the encryption scheme Σ is not the same as the modulus p of the relationships proven in the NIZK.

Corollary 1 (Joint Encryption NIZK [LNP22, Theorem 5.3]). *Let k and q be the public matrix dimension and modulus of Σ , $k' = 2k + 1$ the size of an encryption randomness for Σ in Eq. 3 notations and $B^{(e)}$ the norm bound on RandSpace elements. Let $\psi > 0$ be the approximate range gap and $B^{(d)} > 0$ as both defined in [LNP22, Theorem 5.3], and Π the non-interactive protocol from [LNP22, Theorem 5.3], with p its ring modulus and the constraint $q(B^{(e)}\sqrt{nk'}/2 + 1 + B^{(d)}\psi) < p$. Then Π is a NIZK in the ROM for the relation $\mathbf{R}_\Sigma$, when for $(\mathbf{x} = (\mathbf{c}_1, \mathbf{c}_2, \mathbf{B}_1, \mathbf{B}_2), \mathbf{w} = (\mu, \mathbf{r}_1, \mathbf{r}_2)) \in \mathbf{R}_\Sigma$ we input Π with witness $(\mathbf{r}_1, \mathbf{r}_2, \mu)^t$ and the following statement⁶:*

$$v^{(d)} = v^{(e)} = 2$$

⁵ Binary means all the vector's polynomials' coefficients are either 0 or 1

⁶ Since q is prime, its inverse is well-defined modulo p as long as q does not divide p .

$$\begin{aligned}
\mathbf{E}_1^{(d)} &= \frac{1}{q} \begin{pmatrix} \mathbf{B}_1 & \mathbf{0}_{k+1,k'} & \mathbf{0}_{k,1} \\ & & \lfloor \frac{q}{2} \rfloor \end{pmatrix}, & \mathbf{v}_1^{(d)} &= \frac{1}{q} \mathbf{c}_1, & \beta_1^{(d)} &= B^{(d)} \\
\mathbf{E}_2^{(d)} &= \frac{1}{q} \begin{pmatrix} \mathbf{0}_{k+1,k'} & \mathbf{B}_2 & \mathbf{0}_{k,1} \\ & & \lfloor \frac{q}{2} \rfloor \end{pmatrix}, & \mathbf{v}_2^{(d)} &= \frac{1}{q} \mathbf{c}_2, & \beta_2^{(d)} &= B^{(d)} \\
\mathbf{E}_1^{(e)} &= (\mathbf{I}_{k'} & \mathbf{0}_{k',k'} & 0), & \mathbf{v}_1^{(e)} &= \mathbf{0}_{k'}, & \beta_1^{(e)} &= B^{(e)} \\
\mathbf{E}_2^{(e)} &= (\mathbf{0}_{k',k'} & \mathbf{I}_{k'} & 0), & \mathbf{v}_2^{(e)} &= \mathbf{0}_{k'}, & \beta_2^{(e)} &= B^{(e)} \\
\mathbf{E}_{bin} &= (\mathbf{0}_{1,k'} & \mathbf{0}_{1,k'} & 1), & \mathbf{v}_{bin} &= 0.
\end{aligned}$$

Equations 5 are instantiated so to check that $\mathbf{r}_1$ and $\mathbf{r}_2$ are in `RandSpace`, while equation 6 is so to check that μ is binary, and thus a proper message. Equations 4 instantiations are a tad more technical, but essentially these aim to ensure the encryption relations hold modulo q , while the NIZK only ensures that the relations above hold modulo p . We defer readers to [LNP22, Sec. 6.3] for full explanations.

Remark 1. Zero-knowledge of the LNP NIZK is proven under the hardness assumption of M-SIS and knowledge soundness is proven under extended M-LWE a variant of M-LWE, see [LNP22, Sec. 2] for more details. Hence, our overall scheme depends on structured lattice hardness assumptions only.

Note that [LNP22] do not prove Π to be one-time simulation sound, which we need for the NY-transform. Hence, we heuristically assume that Π does not only fulfill knowledge soundness, but also one-time simulation soundness. We highlight that the heuristic, that using a NIZK with slightly weaker proved security notions than required in favour of a more efficient scheme does not induce any critical security issues, is rather common. This was for instance done in the recent lattice-based non-interactive key-exchange Swoosh [GdKQ⁺24]. Swoosh used the exactly same LNP NIZK, and heuristically assumed the much stronger property of simulation soundness with additional online-extractability.⁷ We do not see any serious obstacles in proving simulation-soundness. Addressing the formal proof of LNP NIZK's soundness is part of our immediate future directions, which we believe to be beyond the scope of this article.

6 Non-Interference Properties of Rutile

In this section we prove non-interference properties of Rutile defined in Section 4. The non-interference of the key generation is needed to prove probing IND-CPA security in Section 7.1. The non-interference of the decryption is needed as an ingredient of the probing variant of the NY-transform from Section 3.3.

In Sec 4 we described a version of $\text{KeyGen}_{\text{CPA}}$ in which we call the procedure `AddRepNoise`. Essentially, this procedure samples a number of small uniforms — as with `Unif`, — adds them to a d -shared input, adds a constant off-set and

⁷ If we would make the same strong heuristic than Swoosh [GdKQ⁺24], we would only need *one* ciphertext in the NY-transform.

outputs a d -shared variable. In this section we use a slight variation of this procedure which we call $\text{AddRepNoise}_{\text{ER}}$ ⁸ for which small uniforms to be summed are directly given as input and not sampled during the execution of the procedure. We thus call $\text{KeyGen}_{\text{ER}}$ the procedure relying on $\text{AddRepNoise}_{\text{ER}}$ instead of AddRepNoise and which takes as additional secret unmasked input the small uniforms to be input to the two $\text{AddRepNoise}_{\text{ER}}$. For clarity, we note $\rho^{(s)}$ [resp. $\rho^{(e)}$] the part of these small uniforms used to construct $\mathbf{s}$ [resp. $\mathbf{e}$]. It is straightforward that first sampling $\rho^{(s)}$ and $\rho^{(e)}$ and then inputting them $\text{KeyGen}_{\text{ER}}$ is computationally indistinguishable from using $\text{KeyGen}_{\text{CPA}}$ directly. A formal description of $\text{AddRepNoise}_{\text{ER}}$ and $\text{KeyGen}_{\text{ER}}$ can be found in [dPKPR24, Sec. 6], from which most of our inspiration for this section comes from.

In this section we prove existence of a PPT simulator simulating the leakage for up to $(d - 1)$ probes on $\text{KeyGen}_{\text{CPA}}$ and taking as input only the public output of $\text{KeyGen}_{\text{CPA}}$ and up to $(d - 1)$ small uniforms sampled during calls to this AddRepNoise procedure. To do so, we first prove existence of a Nluo simulator for $\text{KeyGen}_{\text{ER}}$, and define the target simulator as the one first sampling up to $(d - 1)$ small uniforms and then inputting them to the Nluo simulator for $\text{KeyGen}_{\text{ER}}$ along with the public key it was forwarded in the beginning. We first prove the $\text{KeyGen}_{\text{ER}}$ is Nluo and then that Dec_{CCA} is Nlo.

Table 1 lists all assumed properties for base procedures, with either a reference to a description of said procedure which matches the indicated property, or with a short description of a NI implementation when non-interference is straightforward (e.g. (bi)linear gadgets).

Table 1: Probing security hypotheses for base procedures

Procedure	Property	Reference (or justification)
$\text{AddRepNoise}_{\text{ER}}$	sNlu	[dPKPR24, Alg. 9]
$\mathbf{A} \times \cdot$	NI	multiply by $\mathbf{A}$ share by share (linear)
$+$	NI	add vectors share by share (bilinear)
$-\mathbf{u}^t \times \cdot$	NI	multiply by $-\mathbf{u}^t$ share by share (linear)
$v+$	NI	add v to one of the shares
1bit-Compress	NI	[BC22, Alg. 15]
Decode	Nlo	[CGMZ23, Cor. 1] applied to [BCRT23, Cor. 5]

Lemma 10. *Under Table 1 assumptions, $\text{KeyGen}_{\text{ER}}$ is Nluo.*

Proof. We denote by $\text{KeyGen}_{\text{ER}}^*$ the circuit in Fig. 6 without the Decode procedure (i.e. the circuit that outputs $\llbracket \mathbf{t} \rrbracket$). Assumptions from Table 1 and design of $\text{KeyGen}_{\text{ER}}^*$ coincides with that of $\text{KeyGen}_{\text{ER}}$ in [dPKPR24, Sec. 6.3], thus, we forwards readers to their proof that $\text{KeyGen}_{\text{ER}}$ is Nlu. By assumption Decode is Nlo, thus, by applying Proposition 2 to composition of $\text{KeyGen}_{\text{ER}}^*$ and Decode, we have the result. $\square$

⁸ ER stands for Explicit Randomness.

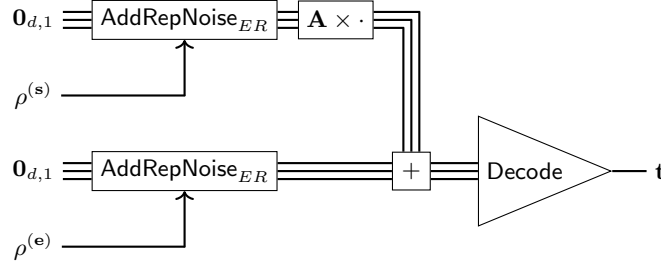


Fig. 6: DAG representing the structure of $\text{KeyGen}_{\text{ER}}$

Lemma 11. *Under Table 1 assumptions, Dec_{CCA} is Nlo.*

Proof. As all computations in Dec_{CCA} are deterministic and depending on public values only except for the single call to the Dec_{CPA} procedure (as defined in Fig. 7), it is sufficient to prove that Dec_{CPA} is Nlo.

The composition of two NI gadgets which only take as input one masked variable and only output one masked variable is NI as well [BBD⁺16, Prop. 4]. All base procedures of Dec_{CPA} are NI except Decode (by assumption), they only take as input one masked variable, only output one masked variable and no public output, therefore, their chain composition (i.e. $[-\mathbf{u}^t \times \cdot] \rightarrow [v + \cdot] \rightarrow [\text{1bit-Compress}]$) is NI as well. As this chain composition is NI it is Nlu as well, and as Decode is Nlo, their composition — i.e. Dec_{CPA} — is Nluo by Proposition 2. As Dec_{CPA} is Nluo and has no unshared input, it is Nlo. $\square$

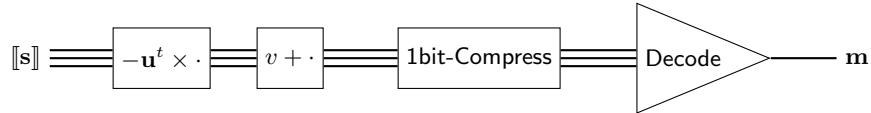


Fig. 7: DAG representing the structure of Dec_{CPA}

7 Probing IND-CCA Security of Topaz

In this last section, we prove the probing IND-CPA security of Rutile. With this at hand, we have now all the ingredients ready to instantiate the NY-transform in the probing setting from Section 3.3. We conclude with concrete parameters for both encryption schemes, Rutile and Topaz.

7.1 Probing IND-CPA Security of Rutile

To prove that the PKE scheme described in Section 4 is probing IND-CPA secure, we proceed through a reduction very similar to the IND-CPA reduction of Kyber

[BDK⁺18]. We need two additional hybrids to replace true leakage of the KeyGen with simulated leakage. The public key together with the simulated leakage can then be captured by the probing M-LWE hardness assumption.

Theorem 2. *Let $\Sigma = (\text{KeyGen}_{\text{CPA}}, \text{Enc}_{\text{CPA}}, \text{Dec}_{\text{CPA}})$ be the scheme described in Section 4 with masking parameter d . We have that Σ is $(d-1)$ -probing IND-CPA and for any PPT adversary $\mathcal{A}$ against Σ with advantage $\text{Adv}_{\mathcal{A}}^{\text{dCPA}}$ we have:*

$$\text{Adv}_{\mathcal{A}}^{\text{dCPA}} \leq \text{Adv}_{\mathcal{R}_q, k, d \cdot (\text{rep}-1)+1, u}^{\text{M-LWE}} + \text{Adv}_{\mathcal{R}_q, k+1, d \cdot \text{rep}, u}^{\text{M-LWE}},$$

where $\text{Adv}^{\text{M-LWE}}$ denotes the advantage against the M-LWE problem.

Proof. The reduction does a sequence of hybrids.

Hybrid₁: The starting hybrid corresponds to the $(d-1)$ -probing IND-CPA original experiment detailed in Figure 3 with bit $b \in \{0, 1\}$.

Hybrid₂: In the second hybrid, we replace the KeyGen function by the following: first we sample $2n \text{ rep}$ small uniforms from $\text{Unif}(\mathcal{R}, k, u)$, and then feed it as input to $\text{KeyGen}_{\text{ER}}$, where the latter is as described in Section 6. Without loss of generality we only consider probes on $\text{KeyGen}_{\text{ER}}$. Indeed, all probable wires of Unif are part of the input wires of $\text{KeyGen}_{\text{ER}}$, thus, considering probes on $\text{KeyGen}_{\text{ER}}$ does not prevent from leaking values from Unif .

Hybrid₃: Lemma 10 proves that $\text{KeyGen}_{\text{ER}}$ is Nluc. In the third hybrid we use this property to replace the real leakage output by $\text{KeyGen}_{\text{ER}}^{\text{S}}$ with simulated leakage output by $\text{KeyGen}_{\text{ER}}$'s Nluc simulator. Let $\text{SimProb}, \text{SimLeak}$ be the PPT Nluc simulators of $\text{KeyGen}_{\text{ER}}$ (see. Section 6) and $\overline{\mathcal{P}}_{KG}$ any set of probes forwarded by the adversary. We denote by $(\rho_i)_i$ the small uniforms carried by the wires specified by the output of $\text{SimProb}(\overline{\mathcal{P}}_{KG})$ and denote by pk the public key output by $\text{KeyGen}_{\text{ER}}$. We now replace real leakage $\mathcal{L}_{KG}$ output by $\text{KeyGen}_{\text{ER}}^{\text{S}}$ by simulated leakage $\mathcal{L}'_{KG}$ output by $\text{SimLeak}((\rho_i), \text{pk}, \overline{\mathcal{P}})$.

Hybrid₄: In the fourth hybrid, we replace the pk output by $\text{KeyGen}_{\text{ER}}$ by a pair $(\mathbf{A}, \mathbf{u}) \leftarrow \mathcal{R}_q^{k \times k} \times \mathcal{R}_q^k$ uniformly sampled.

Hybrid₅: In the fifth hybrid, we replace the ciphertext ct^* by a vector $\mathbf{c} \leftarrow \mathcal{R}_q^{k+1}$ uniformly sampled.

We now prove the advantages for distinguishing between consecutive hybrids.

Lemma 12 (Hybrid₁ to Hybrid₂). *The advantage in distinguishing Hybrid₁ and Hybrid₂ is zero.*

Proof. The only difference between the two hybrids is that small uniforms are sampled earlier in Hybrid₂, this does not induce any change in either pk , $\mathcal{L}_{KG}$, or ct^* . Hence, the adversary's advantage is the same. ■

Lemma 13 (Hybrid₂ to Hybrid₃). *The advantage in distinguishing Hybrid₂ and Hybrid₃ is zero.*

Proof Sketch. The overall idea is to use the Nluc property of $\text{KeyGen}_{\text{ER}}$ to prove that the joint distribution of the public key and leakage is the same in both hybrids. If so, the views of $\mathcal{A}$ are identical in both hybrids, thus their distinguishing advantage is zero. The Nluc property implies identical distribution of the leakages under the assumption that public output — i.e the public key — and all input — i.e small uniforms — are fixed. But in our hybrids those values come from certain distributions. We account for this by using Bayes formula (to condition on fixed values for the public key) and by using the law of total probability (to condition on possible small uniform values). Now, we can invoke the Nluc property and the proof ensues. We note it is important that in both hybrids this set of values for small uniforms is the same, as the sampling of small uniforms and public key computation is done the same way. A full proof can be found in App. D.

Lemma 14 (Hybrid₃ to Hybrid₄). *For any PPT adversary $\mathcal{A}$, the advantage in distinguishing Hybrid₃ and Hybrid₄ is upper bounded by $\text{Adv}_{\mathcal{R}_q, k, d, (\text{rep}-1)+1, u}^{\text{M-LWE}}$.*

Proof. Let $\mathcal{A}$ be a PPT adversary distinguishing between Hybrid₃ and Hybrid₄ with advantage at least Adv , then we build a PPT adversary $\mathcal{B}$ that solves the $(d-1)$ -probing M-LWE $_{\mathcal{R}_q, d, k, d \cdot \text{rep}, u}$ problem with advantage at least Adv .

We begin by describing the reduction. First $\mathcal{B}$ receives as input a matrix $\mathbf{A}$. Upon receiving $\overline{\mathcal{P}}_{KG}$ from $\mathcal{A}$, the reduction $\mathcal{B}$ sends $\overline{\mathcal{P}} := \text{SimProb}(\overline{\mathcal{P}}_{KG})$ as the probes on small uniforms for the $(d-1)$ -probing M-LWE instance. The reduction $\mathcal{B}$ then receives the probing M-LWE instance $(\mathbf{b}, \mathcal{L})$. They compute the leakage $\mathcal{L}_{KG} = \text{SimLeak}(\mathcal{L}, \mathbf{b}, \overline{\mathcal{P}}_{KG})$ and send it to $\mathcal{A}$ along with $\text{pk} = (\mathbf{A}, \mathbf{b})$. The reduction $\mathcal{B}$ outputs as their guess $b' = 0$ if $\mathcal{A}$ guesses Hybrid₃, and $b' = 1$ if $\mathcal{A}$ guesses Hybrid₄.

When $\mathcal{B}$ is input with an M-LWE instance and real leakage, the public key pk and leakage $\mathcal{L}_{KG}$ are computed exactly as in Hybrid₃. When they are input with a uniform vector and fake leakage, the public key pk and leakage $\mathcal{L}_{KG}$ are computed exactly as in Hybrid₄.

As in both situations $\mathcal{B}$ computes either hybrid exactly as they are described, it follows that whenever $\mathcal{A}$ wins, the adversary $\mathcal{B}$ also wins. This concludes the reduction. By Proposition 1, we have that $(d-1)$ -probing M-LWE $_{\mathcal{R}_q, d, k, d \cdot \text{rep}, u} \Leftrightarrow \text{M-LWE}_{\mathcal{R}_q, k, d \cdot (\text{rep}-1)+1, u}$. This conclude the proof. ■

Lemma 15 (Hybrid₄ to Hybrid₅). *The advantage in distinguishing Hybrid₄ and Hybrid₅ is upper bounded by $\text{Adv}_{\mathcal{R}_q, k+1, d \cdot \text{rep}, u}^{\text{M-LWE}}$.*

Proof. As in both hybrids the leakage is now independent from pk and ct^* , we are in the same situation than in Kyber's reduction when distinguishing between games G_1 and G_2 [BDK⁺18, Thm. 2]. This concludes the proof. ■

□

7.2 Putting Everything Together

We have now all the pieces to put everything together for constructing our probing IND-CCA secure encryption scheme, which we call Topaz.

Recall from Theorem 1 that we need two things. First, an encryption scheme Σ which is correct and probing IND-CPA secure, whose decryption is Nlo and which is NY-compatible for RandSpace and RandDistr. Second, a non-interactive proof system Π for the relation R_Σ which is complete, zero-knowledge and one-time simulation sound.

Regarding the first item, we described a lattice-based instantiation of Σ in Section 4, proved it probing IND-CPA secure under standard M-LWE in Theorem 2 and showed its decryption algorithm to be Nlo in Lemma 11. Moreover, we proved it NY-compatible in Lemma 9.

Regarding the second item, we argued in Section 5 that the lattice-based NIZK from [LNP22] is expressive enough to proof R_Σ and (heuristically) argued its one-time simulation soundness.

7.3 Estimates for Rutile and Topaz

In this section, we provide non-optimized sets of parameters for Rutile and Topaz. We expect all sizes to be reducible, for instance using ciphertext compression or better encoding strategies for the secret key.

Rutile Parameters: For the public key, to be closer to practice, we consider $\text{pk} = (\mathbf{t}, \text{seed})$ instead of $\text{pk} = (\mathbf{t}, \mathbf{A})$. Thus, $|\text{pk}| = nk \cdot \lceil \log_2 q \rceil + \kappa$, $|\text{sk}| = nk \cdot \lceil \log_2(d \text{ rep}) \rceil$ and $|\text{ct}| = (k+1)n \cdot \lceil \log_2 q \rceil$, where $|\cdot|$ denotes the bit size.

An attractive feature of our masking-friendly encryption scheme is that the ring degree $n = 256$, the matrix dimension $k = 3$, the bound for the sum of uniforms $u = 2$ and the chosen 24-bit modulus $q = 11017217$ are independent from the masking level (number of shares). The only parameters that change are the number of shares d and the parameter rep , which are chosen such that the product $d \cdot \text{rep} = 2^5$ remains constant. Note that by the correctness Lemma 8, we require $q > 4 \cdot 2^{2u} \cdot d \text{ rep} \cdot n(6k+3)$. This restriction depends on the maximal masking level d we want to support. For q , we choose the smallest which fully splits in $\mathcal{R}$, that is $q = 1 \bmod 2n$.

Overall, the size of the secret/public keys and that of the cipher text remain constant as d increases. We have $|\text{sk}| = 0.48 \text{ KB}$, $|\text{pk}| = 2.32 \text{ KB}$, $|\text{ct}| = 3.07 \text{ KB}$.

Using these parameters, we estimate the M-LWE security level using the Lattice Estimator [APS15] with zero-centered secret and error distributions of variance given by $\sigma = 2^u \sqrt{l/6}$, where $l = d(\text{rep} - 1) + 1$ the latter coming from Theorem 2. Note that σ slowly decreases as d increases. Therefore, the security levels returned by the Lattice Estimator also slightly decreases with d , ranging from $\kappa = 141$ for $d = 1$ (i.e. no masking) to $\kappa = 138$ for $d = 8$.

Topaz Parameters: Note that a secret key of Topaz is exactly a secret key of Rutile. The size of the public keys of Topaz is doubled as we need to store two public keys of Rutile. And finally, the resulting ciphertext size is twice the size of Rutile ciphertexts plus the NIZK proof size. That is: $|\text{sk}| = 0.48 \text{ KB}$, $|\text{pk}| = 4.64 \text{ KB}$, $|\text{ct}| = 33.4 \text{ KB}$.

To estimate the size of the NIZK proof, we adapted the sage script for [LNP22, Sec. 6.3] accessible at <https://github.com/khanhcrypto/LBZKP/blob/main/>

`ver_enc_param_set_1.sage` to our joint encryption setting. We use the formula of [LNP22, Sec. 6.3] and our norm bound for encryption randomness B to obtain the lower bound of 53 on the bit-size of the NIZK modulus p . Our adapted script can be found as supplementary material. The final security level of **Topaz** is the minimum of the security of Rutile and the used NIZK, the latter being set to 128.

7.4 Comparison with Kyber

The time overhead of masking **Topaz** is asymptotically quadratic in the number of shares due to the 1-bit compression during decryption. For **Kyber**, it is also quadratic due to the use of this same operation, as well multiple A2B/B2A mask-conversion gadgets and multiple occurrences of PRF's [BC22, BGR⁺21, CGMZ23, CGMZ22]. Therefore, we believe masking **Topaz** to have a significantly smaller concrete time overhead than **Kyber**, as we reduced the bottleneck to a single operation: the 1-bit compression. Avoiding the FO comes with other advantages. Proving a scheme to be secure in the probing model allows to have a "minimum baseline" of side-channel security, but there remains a gap between the proof model and actual side-channel resistance. The FO transform adds operations depending on the secret key, creating a big attack surface, even when proven secure in the probing model. This is not the case of the NY transform, whose NIZK verification is secret-independent. Hence, we believe NY-transformed schemes to be more side-channel resistant than FO-transformed schemes. Also, as masking is added to **Kyber**'s finished design, it only leads to a time/memory overhead, and do not impact the ciphertext size. Our approach was to modify the design, paying with a significantly larger ciphertext, in order to get a lighter time/memory overhead, and more importantly, a significantly smaller attack surface, as we avoid the FO-transform. Besides, the size of lattice-based NIZK's has greatly improved in the last decade, and any further improvement will reduce the ciphertext sizes of **Topaz**. While **Topaz** ciphertexts are significantly larger than **Kyber**'s, it will thus benefit from ongoing research on lattice-based compact NIZKs. We leave concrete benchmarking, as well as performing side-channel attacks on **Topaz** as interesting future works.

Acknowledgments

We thank our Raccoon discussion group, especially Thomas Roche, for their helpful feedback. This work was supported by the French government through the National Research Agency, under the projects ANR-21-ASTR-0016 *AMIRAL*, ANR-22-PECY-003 *SecureCompute*, and ANR-25-CE39-4214-01 *RELATE*.

References

- AASA⁺20. G. Alagic, J. Alperin-Sheriff, D. Apon, D. Moody, et al. Status report on the second round of the NIST post-quantum cryptography standardization process. Technical Report NISTIR 8309, National Institute of Standards and Technology, July 2020. Available from the NIST website.

- ABC⁺24. G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, N. Waller, and Y.-K. Liu. Status report on the first round of the additional digital signature schemes for the NIST post-quantum cryptography standardization process. Technical Report NISTIR 8528, National Institute of Standards and Technology, October 2024. “Final” version, published October 24, 2024.
- AGV09. A. Akavia, S. Goldwasser, and V. Vaikuntanathan. Simultaneous hardcore bits and cryptography against memory attacks. In O. Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 474–495. Springer, Berlin, Heidelberg, March 2009.
- APS15. M. R. Albrecht, R. Player, and S. Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.
- BBD⁺16. G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, P.-Y. Strub, and R. Zucchini. Strong non-interference and type-directed higher-order masking. In E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, editors, *ACM CCS 2016*, pages 116–129. ACM Press, October 2016.
- BBE⁺18. G. Barthe, S. Belaïd, T. Espitau, P.-A. Fouque, B. Grégoire, M. Rossi, and M. Tibouchi. Masking the GLP lattice-based signature scheme at any order. In J. B. Nielsen and V. Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 354–384. Springer, Cham, April / May 2018.
- BC22. O. Bronchain and G. Cassiers. Bitslicing arithmetic/boolean masking conversions for fun and profit with application to lattice-based KEMs. *IACR TCHES*, 2022(4):553–588, 2022.
- BCRT23. S. Belaïd, G. Cassiers, M. Rivain, and A. R. Taleb. Unifying freedom and separation for tight probing-secure composition. In H. Handschuh and A. Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 440–472. Springer, Cham, August 2023.
- BDK⁺18. J. W. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. CRYSTALS - Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy*, pages 353–367. IEEE Computer Society Press, April 2018.
- BGR⁺21. J. W. Bos, M. Gourjon, J. Renes, T. Schneider, and C. van Vredendaal. Masking Kyber: First- and higher-order implementations. *IACR TCHES*, 2021(4):173–214, 2021.
- BGV12. Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In S. Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012.
- BJRW23. K. Boudgoust, C. Jeudy, A. Roux-Langlois, and W. Wen. On the hardness of module learning with errors with short distributions. *Journal of Cryptology*, 36(1):1, January 2023.
- BRR25. S. Belaïd, M. Rivain, and M. Rossi. New techniques for random probing security and application to raccoon signature scheme. In S. Fehr and P.-A. Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 94–123. Springer, Cham, May 2025.
- CGMZ21. J.-S. Coron, F. Gérard, S. Montoya, and R. Zeitoun. High-order table-based conversion algorithms and masking lattice-based encryption. Cryptology ePrint Archive, Report 2021/1314, 2021.

- CGMZ22. J.-S. Coron, F. Gérard, S. Montoya, and R. Zeitoun. High-order table-based conversion algorithms and masking lattice-based encryption. *IACR TCHES*, 2022(2):1–40, 2022.
- CGMZ23. J.-S. Coron, F. Gérard, S. Montoya, and R. Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR TCHES*, 2023(1):153–192, 2023.
- CGTZ23. J.-S. Coron, F. Gérard, M. Trannoy, and R. Zeitoun. Improved gadgets for the high-order masking of Dilithium. *IACR TCHES*, 2023(4):110–145, 2023.
- DDF14. A. Duc, S. Dziembowski, and S. Faust. Unifying leakage models: From probing attacks to noisy leakage. In P. Q. Nguyen and E. Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 423–440. Springer, Berlin, Heidelberg, May 2014.
- dEK⁺23. R. del Pino, T. Espitau, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, M. Rossi, and M. Saarinen. Raccoon. Technical report, National Institute of Standards and Technology, 2023. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.
- DKM⁺24. R. Del Pino, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, and M.-J. O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In M. Joye and G. Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.
- dPEK⁺23. R. del Pino, T. Espitau, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, M. Rossi, and M.-J. Saarinen. Raccoon: A side-channel secure signature scheme, 2023.
- dPKPR24. R. del Pino, S. Katsumata, T. Prest, and M. Rossi. Raccoon: A masking-friendly signature proven in the probing model. In L. Reyzin and D. Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 409–444. Springer, Cham, August 2024.
- EEN⁺24. M. F. Esgin, T. Espitau, G. Niot, T. Prest, A. Sakzad, and R. Steinfeld. Plover: Masking-friendly hash-and-sign lattice signatures. In M. Joye and G. Leander, editors, *EUROCRYPT 2024, Part VII*, volume 14657 of *LNCS*, pages 316–345. Springer, Cham, May 2024.
- FRW25. T. Feneuil, M. Rivain, and A. Warmé-Janville. Masking-friendly post-quantum signatures in the threshold-computation-in-the-head framework. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(4):667–710, 2025.
- GdKQ⁺24. P. Gajland, B. de Kock, M. Quaresma, G. Malavolta, and P. Schwabe. SWOOSH: Efficient lattice-based non-interactive key exchange. In D. Balzarotti and W. Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.
- HHMS25. K. Hövelmanns, A. Hülsing, C. Majenz, and F. Sisinni. (Un)breakable curses - re-encryption in the Fujisaki-Okamoto transform. In S. Fehr and P.-A. Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 245–274. Springer, Cham, May 2025.
- HLM⁺23. C. Hoffmann, B. Libert, C. Momin, T. Peters, and F.-X. Standaert. POLKA: Towards leakage-resistant post-quantum CCA-secure public key encryption. In A. Boldyreva and V. Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 114–144. Springer, Cham, May 2023.
- HPA21. J. Howe, T. Prest, and D. Apon. SoK: How (not) to design and implement post-quantum cryptography. In K. G. Paterson, editor, *CT-RSA 2021*, volume 12704 of *LNCS*, pages 444–477. Springer, Cham, May 2021.

- HPS98. J. Hoffstein, J. Pipher, and J. H. Silverman. NTRU: A ring-based public key cryptosystem. In *Third Algorithmic Number Theory Symposium (ANTS)*, volume 1423 of *LNCS*, pages 267–288. Springer, June 1998.
- HSST23. J. Hermelink, S. Streit, E. Strieder, and K. Thieme. Adapting belief propagation to counter shuffling of NTTs. *IACR TCHES*, 2023(1):60–88, 2023.
- ISW03. Y. Ishai, A. Sahai, and D. Wagner. Private circuits: Securing hardware against probing attacks. In D. Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, Berlin, Heidelberg, August 2003.
- KJJ99. P. C. Kocher, J. Jaffe, and B. Jun. Differential power analysis. In M. J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 388–397. Springer, Berlin, Heidelberg, August 1999.
- Koc96. P. C. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In N. Kobitz, editor, *CRYPTO’96*, volume 1109 of *LNCS*, pages 104–113. Springer, Berlin, Heidelberg, August 1996.
- KR19. Y. T. Kalai and L. Reyzin. A survey of leakage-resilient cryptography. In *Providing Sound Foundations for Cryptography*, pages 727–794. ACM, 2019.
- LDK⁺22. V. Lyubashevsky, L. Ducas, E. Kiltz, T. Lepoint, P. Schwabe, G. Seiler, D. Stehlé, and S. Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- Lin06. Y. Lindell. A simpler construction of CCA2-secure public-key encryption under general assumptions. *Journal of Cryptology*, 19(3):359–377, July 2006.
- LNP22. V. Lyubashevsky, N. K. Nguyen, and M. Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 71–101. Springer, Cham, August 2022.
- LPR13. V. Lyubashevsky, C. Peikert, and O. Regev. A toolkit for ring-LWE cryptography. In T. Johansson and P. Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 35–54. Springer, Berlin, Heidelberg, May 2013.
- LS15. A. Langlois and D. Stehlé. Worst-case to average-case reductions for module lattices. *DCC*, 75(3):565–599, 2015.
- McE78. R. J. McEliece. A public-key cryptosystem based on algebraic coding theory. The deep space network progress report 42-44, Jet Propulsion Laboratory, California Institute of Technology, January/February 1978. https://ipnpr.jpl.nasa.gov/progress_report2/42-44/44N.PDF.
- NS09. M. Naor and G. Segev. Public-key cryptosystems resilient to key leakage. In S. Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 18–35. Springer, Berlin, Heidelberg, August 2009.
- NY90. M. Naor and M. Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd ACM STOC*, pages 427–437. ACM Press, May 1990.
- oST22. N. I. of Standards and Technology. Call for additional digital signature algorithms for the NIST post-quantum cryptography standardization process. Technical report, National Institute of Standards and Technology, September 2022. Call for proposals for additional post-quantum signature algorithms (Round 4).

- oST24. N. I. of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard (crystals-kyber). Technical report, National Institute of Standards and Technology, August 2024. Draft Federal Information Processing Standard (FIPS 203).
- Pat96. J. Patarin. Hidden fields equations (HFE) and isomorphisms of polynomials (IP): Two new families of asymmetric algorithms. In U. M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 33–48. Springer, Berlin, Heidelberg, May 1996.
- PPM17. R. Primas, P. Pessl, and S. Mangard. Single-trace side-channel attacks on masked lattice-based encryption. In W. Fischer and N. Homma, editors, *CHES 2017*, volume 10529 of *LNCS*, pages 513–533. Springer, Cham, September 2017.
- PPS26. D. Pay, T. Peters, and F.-X. Standaert. In mid-stream: Removing the FO-transform helps against leakage but is not enough. Cryptology ePrint Archive, Paper 2026/157, 2026.
- Reg05. O. Regev. On lattices, learning with errors, random linear codes, and cryptography. In H. N. Gabow and R. Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.
- Sho94. P. W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th FOCS*, pages 124–134. IEEE Computer Society Press, November 1994.
- UXT⁺22. R. Ueno, K. Xagawa, Y. Tanaka, A. Ito, J. Takahashi, and N. Homma. Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. *IACR TCHES*, 2022(1):296–322, 2022.
- WBD24. R. Wang, M. Brisfors, and E. Dubrova. A side-channel attack on a higher-order masked CRYSTALS-Kyber implementation. In C. Pöpper and L. Batina, editors, *ACNS 2024, Part III*, volume 14585 of *LNCS*, pages 301–324. Springer, Cham, March 2024.

Appendix A Missing Proofs of Section 2

A.1 Proof of Lemma 1

Proof. We have by triangular inequality that $\|\mathbf{a}^t \mathbf{b}\|_\infty \leq \sum_{i=1}^k \|\mathbf{a}_i \cdot \mathbf{b}_i\|_\infty$. The polynomial product $\mathbf{a}_i \cdot \mathbf{b}_i$ can be represented as a matrix-vector product and applying the Cauchy-Schwarz inequality on every row of the matrix independently gives $\|\mathbf{a}_i \cdot \mathbf{b}_i\|_\infty \leq \|\mathbf{a}_i\|_2 \cdot \|\mathbf{b}_i\|_2$. Here, the shape of power-of-two cyclotomics is important as it yields nega-cyclic matrices which do not increase the norms. Thus, we have $\|\mathbf{a}^t \mathbf{b}\|_\infty \leq \sum_{i=1}^k \|\mathbf{a}_i\|_2 \cdot \|\mathbf{b}_i\|_2$. This sum itself is an inner product. Applying Cauchy-Schwarz inequality a second time thus yields the proof. $\square$

A.2 Proof of Proposition 1

Proof. " $\Rightarrow$ " We start by describing the reduction. Let $\mathcal{A}$ be an adversary in the M-LWE experiment from Figure 1a and let $\mathcal{B}$ be an adversary in the experiment of $(d-1)$ -probing M-LWE from Figure 1b. Let $\mathcal{A}$ be given as input $(\mathbf{A}, \mathbf{b})$. They forward $\mathbf{A}$ as first input to $\mathcal{B}$. Upon receiving $\overline{\mathcal{P}} \subsetneq \{1, \dots, l\}$ as the probe indices requested by $\mathcal{B}$, the adversary $\mathcal{A}$ verifies that $|\overline{\mathcal{P}}| < d$, else they abort the experiment. Let $\widetilde{\mathcal{P}}$ denote an arbitrary subset of $\{1, \dots, l\}$ of size exactly $d-1$ containing $\overline{\mathcal{P}}$. Then, for all $i \in \widetilde{\mathcal{P}}$, the adversary $\mathcal{A}$ samples $\widetilde{\rho}_i^{(s)} \leftarrow \text{Unif}(\mathcal{R}, k, u)$ and $\widetilde{\rho}_i^{(e)} \leftarrow \text{Unif}(\mathcal{R}, k, u)$ and sets

$$\widetilde{\mathbf{b}} = \mathbf{b} + \mathbf{A} \left(\sum_{i \in \widetilde{\mathcal{P}}} \widetilde{\rho}_i^{(s)} \right) + \left(\sum_{i \in \widetilde{\mathcal{P}}} \widetilde{\rho}_i^{(e)} \right) \quad \text{and} \quad \mathcal{L} = \left(\widetilde{\rho}_i^{(s)}, \widetilde{\rho}_i^{(e)} \right)_{i \in \overline{\mathcal{P}}}.$$

The reduction $\mathcal{A}$ then gives $(\widetilde{\mathbf{b}}, \mathcal{L})$ as input to $\mathcal{B}$. Upon receiving the guess b' from $\mathcal{B}$, the adversary $\mathcal{A}$ forwards b' as their own guess.

We now argue that the advantage of $\mathcal{A}$ winning the $\text{M-LWE}_{\mathcal{R}_q, k, (l-d+1), u}$ experiment is the same as the advantage of $\mathcal{B}$ winning the $(d-1)$ -probing $\text{M-LWE}_{\mathcal{R}_q, d, k, l, u}$ experiment. First, note that the matrix $\mathbf{A}$ has the same distribution in both experiments. If $b = 0$ in the M-LWE experiment, we know that $\mathbf{b}$ can be written as $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where both $\mathbf{s}$ and $\mathbf{e}$ are composed of $l-d+1$ summands. Given that $|\{1, \dots, l\} \setminus \overline{\mathcal{P}}| = l-d+1$, we can rearrange and rename the $l-d+1$ summands of $\mathbf{s}$ and $\mathbf{e}$ to $\left(\widetilde{\rho}_i^{(s)}, \widetilde{\rho}_i^{(e)} \right)_{i \in \{1, \dots, l\} \setminus \overline{\mathcal{P}}}$. Now, it yields

$$\widetilde{\mathbf{b}} = \mathbf{A}\mathbf{s} + \mathbf{e} + \mathbf{A} \left(\sum_{i \in \widetilde{\mathcal{P}}} \widetilde{\rho}_i^{(s)} \right) + \left(\sum_{i \in \widetilde{\mathcal{P}}} \widetilde{\rho}_i^{(e)} \right) = \mathbf{A} \left(\sum_{i=1}^l \widetilde{\rho}_i^{(s)} \right) + \left(\sum_{i=1}^l \widetilde{\rho}_i^{(e)} \right).$$

Hence, $(\widetilde{\mathbf{b}}, \mathcal{L})$ has the correct distribution for the case $b = 0$ in the probing experiment. On the other hand, if $b = 1$, $\mathbf{b}$ is uniform, implying the uniformity

of $\tilde{\mathbf{b}}$. Again, $(\tilde{\mathbf{b}}, \mathcal{L})$ has the correct distribution for the case $b = 1$ in the probing experiment. Overall, the cases $b = 0$ and the cases $b = 1$ match in both experiments, implying that the advantage is preserved.

" $\Leftarrow$ " As before, we begin with describing the reduction. Let $\mathcal{B}$ be an adversary in the $(d-1)$ -probing M-LWE experiment from Figure 1b and let $\mathcal{A}$ be an adversary in the experiment of M-LWE from Figure 1a. Let $\mathcal{B}$ be given as first input $\mathbf{A}$. They output $\overline{\mathcal{P}} = \{l-d+2, \dots, l\}$ which is a set of size $d-1$. Let $(\mathbf{b}, \mathcal{L})$ be the second input given to $\mathcal{B}$, where $\mathcal{L} = (\rho_i^{(s)}, \rho_i^{(e)})_{i \in \overline{\mathcal{P}}}$. They set $\tilde{\mathbf{b}} = \mathbf{b} - \mathbf{A} \left(\sum_{i \in \overline{\mathcal{P}}} \rho_i^{(s)} \right) - \left(\sum_{i \in \overline{\mathcal{P}}} \rho_i^{(e)} \right)$ and provide $(\mathbf{A}, \tilde{\mathbf{b}})$ to $\mathcal{A}$. Upon receiving the guess b' , the adversary $\mathcal{B}$ forwards b' as their own guess.

We argue that the advantage of $\mathcal{B}$ winning the $(d-1)$ -probing M-LWE $_{\mathcal{R}_q, d, n, l, u}$ experiment is the same as the advantage of $\mathcal{A}$ winning the corresponding experiment of M-LWE $_{\mathcal{R}_q, k, (l-d+1), u}$. First, note that the matrix $\mathbf{A}$ has the same distribution in both experiments. If $b = 0$ in the $(d-1)$ -probing M-LWE experiment,

$$\tilde{\mathbf{b}} = \mathbf{A}\mathbf{s} + \mathbf{e} - \mathbf{A} \left(\sum_{i \in \overline{\mathcal{P}}} \rho_i^{(s)} \right) - \left(\sum_{i \in \overline{\mathcal{P}}} \rho_i^{(e)} \right) = \mathbf{A} \left(\sum_{i=1}^{l-d+1} \rho_i^{(s)} \right) + \left(\sum_{i=1}^{l-d+1} \rho_i^{(e)} \right).$$

Hence, $(\mathbf{A}, \tilde{\mathbf{b}})$ has the correct distribution for the case $b = 0$ in the M-LWE experiment. On the other hand, if $b = 1$, $\mathbf{b}$ is uniform, implying the uniformity of $\tilde{\mathbf{b}}$. Again, $(\mathbf{A}, \tilde{\mathbf{b}})$ has the correct distribution for the case $b = 1$ in the M-LWE experiment. Overall, the cases $b = 0$ and the cases $b = 1$ match in both experiments, implying that the advantage is preserved. $\square$

A.3 Proof of Proposition 2

Proof. For simplicity we note $\overline{\mathcal{Z}} := \overline{\mathcal{Y}}_2$, $\overline{\mathcal{Y}} := \overline{\mathcal{Y}}_1 = \overline{\mathcal{X}}_1$ and $\overline{\mathcal{X}} := \overline{\mathcal{X}}_1$, and $\mathcal{Z}$ [resp. $\mathcal{Y}$, $\mathcal{X}$] the values carried by $\overline{\mathcal{Z}}$ [resp. $\overline{\mathcal{Y}}$, $\overline{\mathcal{X}}$]. Also, we note $\overline{\mathcal{Y}}_I = \text{SimProb}_2(\overline{\mathcal{P}}_2)$ [resp. $\overline{\mathcal{Y}}_I = \overline{\mathcal{Y}} \setminus \text{SimProb}_2(\overline{\mathcal{P}}_2)$] and $\mathcal{Y}_I$ [resp. $\mathcal{Y}_I$] the values they carry.

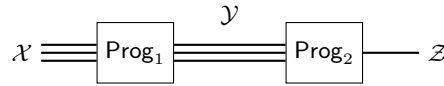


Fig. 8: DAG representing the structure of Prog

Let $\text{SimProb}_1, \text{SimLeak}_1$ be the Nlu simulators for Prog_1 and $\text{SimProb}_2, \text{SimLeak}_2$ the Nlo simulators for Prog_2 . In the following proof, we note $\overline{\mathcal{P}} = (\overline{\mathcal{P}}_1 \cup \overline{\mathcal{P}}_2)$ the set of probes queried on Prog, with $\overline{\mathcal{P}}_1$ probes on Prog_1 computations and $\overline{\mathcal{P}}_2$ probes on Prog_2 computations. By convention, we force $\overline{\mathcal{P}}_1 \cap \overline{\mathcal{P}}_2 = \emptyset$. We note $\mathcal{L}_1$ the values carried by $\overline{\mathcal{P}}_1$ wires and $\mathcal{L}_2$ the values carried by $\overline{\mathcal{P}}_2$ wires.

We prove the result by describing the two simulators SimProb and SimLeak and proving they are Nluo simulators for Prog.

Algorithm 7 SimProb

Input: Sets of probes $\overline{\mathcal{P}}_1, \overline{\mathcal{P}}_2$ on Prog_1 and Prog_2

Output: $\overline{\mathcal{X}}' \subset \overline{\mathcal{X}}$ respecting Def. 14 properties

1: $\overline{\mathcal{Y}}_I \leftarrow \text{SimProb}_2(\overline{\mathcal{P}}_2)$

2: $\overline{\mathcal{X}}' \leftarrow \text{SimProb}_1(\overline{\mathcal{P}}_1 \cup \overline{\mathcal{Y}}_I)$

We show that SimProb match the criteria of the Nluo definition. Let $\overline{\mathcal{P}} = \overline{\mathcal{P}}_1 \cup \overline{\mathcal{P}}_2$ with $|\overline{\mathcal{P}}| < d$. We note $\overline{\mathcal{X}}'_{\text{masked}}$ [resp. $\overline{\mathcal{X}}'_{\text{unmasked}}$] wires of $\overline{\mathcal{X}}'$ carrying masked [resp. unmasked] values. We have that:

- $\overline{\mathcal{X}}'_{\text{masked}} \subset \overline{\mathcal{X}}_{\text{masked}}$ and $\overline{\mathcal{X}}'_{\text{unmasked}} \subset \overline{\mathcal{X}}_{\text{unmasked}}$ by hypothesis on SimProb₁
- $|\llbracket \overline{\mathcal{X}}_I \rrbracket \cap \overline{\mathcal{X}}'_{\text{masked}}| \leq |\overline{\mathcal{P}}_1 \cup \overline{\mathcal{Y}}_I| \leq |\overline{\mathcal{P}}_1| + |\overline{\mathcal{Y}}_I| \leq |\overline{\mathcal{P}}_1| + |\overline{\mathcal{P}}_2| = |\overline{\mathcal{P}}|$, where first inequality comes from hypothesis on SimProb₁, second comes from triangular inequality and last one comes from hypothesis on SimProb₂.
- $|\overline{\mathcal{X}}'_{\text{unmasked}}| \leq |\overline{\mathcal{P}}_1 \cup \overline{\mathcal{Y}}_I| \leq |\overline{\mathcal{P}}_1| + |\overline{\mathcal{Y}}_I| \leq |\overline{\mathcal{P}}_1| + |\overline{\mathcal{P}}_2| = |\overline{\mathcal{P}}|$, where first inequality comes from hypothesis on SimProb₁, second comes from triangular inequality and last one comes from hypothesis on SimProb₂.

Thus SimProb meets the Nluo criteria.

Algorithm 8 SimLeak

Input: Public output $\mathcal{Y}_{\text{public}}$, some input values $\mathcal{X}'$ and set of probes $\overline{\mathcal{P}} = (\overline{\mathcal{P}}_1 \cup \overline{\mathcal{P}}_2)$

Output: l_1, l_2 simulated values for $\overline{\mathcal{P}}_1, \overline{\mathcal{P}}_2$

1: $\overline{\mathcal{Y}}_I \leftarrow \text{SimProb}_2(\overline{\mathcal{P}}_2)$

2: $l_1, y_I \leftarrow \text{SimLeak}_1(\mathcal{X}', \overline{\mathcal{P}}_1 \cup \overline{\mathcal{Y}}_I)$

$\triangleright y_I$ simulates values carried by $\overline{\mathcal{Y}}_I$

3: $l_2 \leftarrow \text{SimLeak}_2(y_I, \mathcal{Y}_{\text{public}}, \overline{\mathcal{P}}_2)$

We now show that SimLeak meets the criteria of the Nluo definition. We first state a claim on the SimLeak₁ and SimLeak₂ simulators.

Claim. Let l_1, l_2 values $\overline{\mathcal{P}}_1$ and $\overline{\mathcal{P}}_2$ could carry. For any y_I and $y_{\overline{I}}$ some values $\overline{\mathcal{Y}}_I$ and $\overline{\mathcal{Y}}_{\overline{I}}$ could carry, any input values $\mathcal{X}$ and any public output values $\mathcal{Z}$, we have the two following equalities:

$$\Pr(\mathcal{L}_1 = l_1, \mathcal{Y}_I = y_I \mid \mathcal{X}, \mathcal{Z}) = \Pr(\text{SimLeak}_1() = (l_1, y_I) \mid \mathcal{X}, \mathcal{Z})$$

$$\Pr(\mathcal{L}_2 = l_2 \mid \mathcal{X}, \mathcal{Z}, y_I, y_{\overline{I}}, l_1) = \Pr(\text{SimLeak}_2() = l_2 \mid \mathcal{X}, \mathcal{Z}, \text{SimLeak}_1() = (l_1, y_I))$$

where we abbreviate $\text{SimLeak}_2(y_I, \mathcal{Z}, \overline{\mathcal{P}}_2)$ into $\text{SimLeak}_2()$ and $\text{SimLeak}_1(\mathcal{X}', \overline{\mathcal{P}}_1 \cup \overline{\mathcal{Y}}_I)$ into $\text{SimLeak}_1()$ for clarity of notation.

This claim stems from the Nlu/Nlo properties of SimLeak₁ and SimLeak₂, and from the fact that $\mathcal{Z}$ is deterministic in $\mathcal{X}$. We highlight that if $\mathcal{Z}$ is not deterministic in $\mathcal{X}$ then first equality does not hold, but since it is, the dependence

in $\mathcal{Z} := f(\mathcal{X})$ is already implicitly set by the dependence on $\mathcal{X}$ and the equality simply is the formulation of a **Nlu** property for **Prog₁**.

We now need argue that **SimLeak** output has the same distribution than joint distribution $(\mathcal{L}_1, \mathcal{L}_2)$ for a given set of probes $\overline{\mathcal{P}} = \overline{\mathcal{P}}_1 \cup \overline{\mathcal{P}}_2$ (with $\overline{\mathcal{P}}_1 \cap \overline{\mathcal{P}}_2 = \emptyset$), a given input $\mathcal{X}$ and public output $\mathcal{Z} := f(\mathcal{X})$. Let l_1, l_2 values $\overline{\mathcal{P}}_1$ and $\overline{\mathcal{P}}_2$ could carry, we have:

$$\begin{aligned}
& \Pr(\mathcal{L}_1 = l_1, \mathcal{L}_2 = l_2 | \mathcal{X}, \mathcal{Z}) \\
&= \sum_{y \in \mathcal{S}_{\overline{\mathcal{Y}}_1}} \Pr(\mathcal{L}_1 = l_1, \mathcal{L}_2 = l_2, \mathcal{Y} = y | \mathcal{X}, \mathcal{Z}) \\
&= \sum_{y \in \mathcal{S}_{\overline{\mathcal{Y}}_1}} \Pr(\mathcal{L}_1 = l_1, \mathcal{Y} = y | \mathcal{X}, \mathcal{Z}) \cdot \Pr(\mathcal{L}_2 = l_2 | \mathcal{X}, \mathcal{Z}, \mathcal{L}_1 = l_1, \mathcal{Y} = y) \\
&= \sum_{y_I \in \mathcal{S}_{\overline{\mathcal{Y}}_I}} \sum_{y_{\bar{I}} \in \mathcal{S}_{\overline{\mathcal{Y}}_{\bar{I}}}} \Pr(\mathcal{L}_1 = l_1, \mathcal{Y} = y | \mathcal{X}, \mathcal{Z}) \cdot \Pr(\mathcal{L}_2 = l_2 | \mathcal{X}, \mathcal{Z}, \mathcal{L}_1 = l_1, \mathcal{Y} = y) \\
&= \sum_{y_I \in \mathcal{S}_{\overline{\mathcal{Y}}_I}} \sum_{y_{\bar{I}} \in \mathcal{S}_{\overline{\mathcal{Y}}_{\bar{I}}}} \Pr(\mathcal{L}_1 = l_1, \mathcal{Y} = y | \mathcal{X}, \mathcal{Z}) \\
&\quad \cdot \Pr(\text{SimLeak}_2() = l_2 \mid \mathcal{X}, \mathcal{Z}, \text{SimLeak}_1() = (l_1, y_I)) \\
&= \sum_{y_I \in \mathcal{S}_{\overline{\mathcal{Y}}_I}} \Pr(\text{SimLeak}_2() = l_2 \mid \mathcal{X}, \mathcal{Z}, \text{SimLeak}_1() = (l_1, y_I)) \\
&\quad \cdot \Pr(\mathcal{L}_1 = l_1, \mathcal{Y}_I = y_I | \mathcal{X}, \mathcal{Z}) \\
&= \sum_{y_I \in \mathcal{S}_{\overline{\mathcal{Y}}_I}} \Pr(\text{SimLeak}_2() = l_2 \mid \mathcal{X}, \mathcal{Z}, \text{SimLeak}_1() = (l_1, y_I)) \\
&\quad \cdot \Pr(\text{SimLeak}_1() = (l_1, y_I) \mid \mathcal{X}, \mathcal{Z}) \\
&= \sum_{y_I \in \mathcal{S}_{\overline{\mathcal{Y}}_I}} \Pr(\text{SimLeak}_2() = l_2, \text{SimLeak}_1() = (l_1, y_I) \mid \mathcal{X}, \mathcal{Z}) \\
&= \Pr(\text{SimLeak}_2() = l_2, \text{SimLeak}_1() = (l_1, \cdot) \mid \mathcal{X}, \mathcal{Z}) \\
&= \Pr(\text{SimLeak}() = l_1, l_2 \mid \mathcal{X}, \mathcal{Z})
\end{aligned}$$

First equality comes from total probability formula, second one comes from Bayes theorem, third one is just a partition of $\overline{\mathcal{Y}}$, forth one comes from our claim, fifth one comes from total probability formula, sixth one comes from our claim, seventh one comes from Bayes theorem, eighth one comes from total probability formula and last one is simply a rephrasing. This concludes our proof. $\square$

A.4 Proof of Proposition 3

Proof. By assumption on **Prog**, we know that its public output $\mathcal{Y}_{public}$ is uniquely determined by the public input and the secret value x whose sharing we input to **Prog**. Let F denote the (deterministic) function such that $\forall x \in \mathcal{R}$, $\mathcal{Y}_{public} := F(x)$. Now let $x \in \mathcal{R}$ a secret value and $\llbracket x \rrbracket$ a privacy-preserving sharing of x .

Since **Prog** is $(d-1)$ -Nlo, there exist **SimProb** and **SimLeak** which for every set of probes $\overline{\mathcal{P}}$ s.t. $|\overline{\mathcal{P}}| < d$, for any input $\mathcal{X} = (\mathcal{X}_{masked}, \emptyset)$ with $\mathcal{X}_{masked} \leftarrow \text{KeyUpdate}(\llbracket x \rrbracket)$ and for public output $\mathcal{Y}_{public} = F(x)$ perfectly simulate $\overline{\mathcal{P}}$ for input $\mathcal{X} = (\mathcal{X}_{masked}, \emptyset)$ and $\mathcal{Y}_{public} := F(x)$, that is

$$\mathcal{L} \stackrel{i.d.}{=} \text{SimLeak}(\mathcal{X}', \mathcal{Y}_{public}, \overline{\mathcal{P}}),$$

where $(\cdot, \mathcal{L}) \leftarrow \text{Prog}^{\mathcal{L}}((\mathcal{X}_{masked}, \emptyset), \overline{\mathcal{P}})$ and $\mathcal{X}'$ are the values of the input wires $\overline{\mathcal{X}}' \leftarrow \text{SimProb}(\overline{\mathcal{P}})$.

By definition of $(d-1)$ -Nlo, it holds $\overline{\mathcal{X}}' = (\overline{\mathcal{X}}'_{masked}, \emptyset)$, with $|\overline{\mathcal{X}}'_{masked}| < d$. Hence, $\mathcal{X}'$ contains at most $d-1$ values of the additive secret sharing $\mathcal{X}_{masked}$ output by **KeyUpdate**($\llbracket x \rrbracket$).

By privacy of the additive secret sharing output by **KeyUpdate**($\llbracket x \rrbracket$), we know that the joint distribution of $(x, \mathcal{X}')$ is identical to the joint distribution of x with $|\mathcal{X}'|$ independently sampled uniform variables over the finite ring $\mathcal{R}$, i.e.:

$$\mathcal{X}' \stackrel{i.d.}{=} (\gamma_i)_{i \in [|\overline{\mathcal{X}}'|]}, \text{ where } \gamma_i \leftarrow_{\$} \mathcal{R}. \quad (7)$$

By application of the probability preservation property to Equation 7 with the function $\text{SimLeak}(\cdot, F(x), \overline{\mathcal{P}})$, we obtain

$$\text{SimLeak}(\mathcal{X}', F(x), \overline{\mathcal{P}}) \stackrel{i.d.}{=} \text{SimLeak}((\gamma_i)_{i \in [|\overline{\mathcal{X}}'|]}, F(x), \overline{\mathcal{P}}).$$

Therefore, we define **Sim** as the PPT simulator that first calls **SimProb** on $\overline{\mathcal{P}}$, then samples a number of independent uniform variables in $\mathcal{R}$ equal to the size of **SimProb**'s output, and then calls **SimLeak** on these independent variables, on $F(x) = \mathcal{Y}_{public}$ ⁹ and on $\overline{\mathcal{P}}$, concluding the proof. $\square$

Appendix B Missing Details of Section 3

B.1 Probing IND-CCA of KEM

As we only need the notion of probing IND-CPA for public-key encryption schemes, we omit an analogue definition for key-encapsulation mechanisms, and directly jump to the notion of probing IND-CCA security. However, if needed, it can be defined in a straight-forward manner.

⁹ in practice, $\mathcal{Y}_{public}$ will be provided to the simulator

Definition 18. A key encapsulation mechanism KEM over key space $\mathcal{K}$ is defined by the following PPT algorithms (KeyGen, Encaps, Decaps), parameterized by the security parameter κ , where:

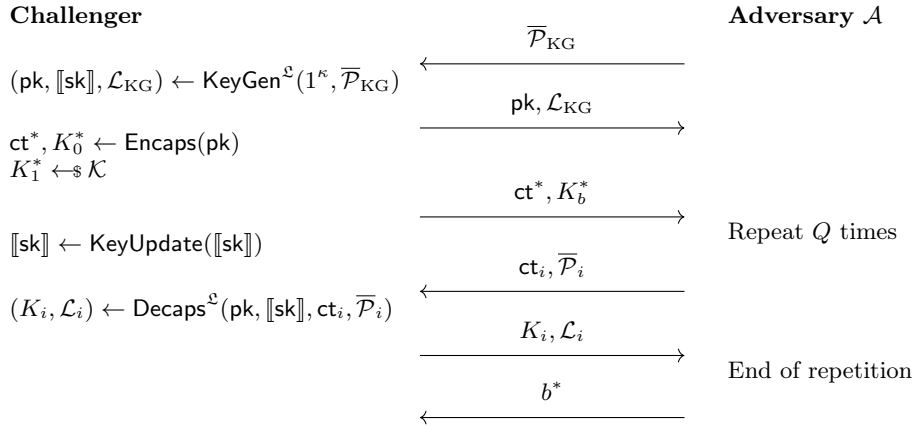
- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$: on input the security parameter κ , the key generation algorithm outputs a public key pk and a secret key sk .
- $(\text{ct}, K) \leftarrow \text{Encaps}(\text{pk})$: on input a public key pk , the encapsulation algorithm outputs a ciphertext ct and a key K .
- $K' \leftarrow \text{Decaps}(\text{pk}, \text{sk}, \text{ct})$: given a public key pk , a secret key sk and a ciphertext ct , the decapsulation algorithm outputs a key $K' \in \mathcal{K}$ or the symbol $\perp$.

Correctness requires that for honestly generated keys $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$, and honest encapsulation $(\text{ct}, K) \leftarrow \text{Encaps}(\text{pk})$, the decapsulation of the ciphertext leads to the same key K (with overwhelming probability over the randomness of KeyGen, Encaps and Decaps). Again, we denote by $\delta_{\text{fail}}(\kappa)$ the decapsulation failure probability.

Definition 19. A KEM scheme is called $(d-1)$ -probing IND-CCA secure if it holds that for all security parameters $\kappa \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$ in the game described in Figure 9 we have:

$$\text{resp. } \text{Adv}_{\text{KEM}}^{\text{dCCA}}(\kappa) := |\Pr[\text{WIN}_{\text{KEM}}^{\text{IND-CCA}}(0)] - \Pr[\text{WIN}_{\text{KEM}}^{\text{IND-CCA}}(1)]| \leq \text{negl}(\kappa).$$

$(d-1)$ -probing IND-CCA for KEM, with $Q = \text{poly}(\kappa)$ and $b \in \{0, 1\}$



$$\text{WIN}_{\text{KEM}}^{\text{IND-CCA}}(b) = (b = b^*) \wedge (\text{ct}^* \notin \{\text{ct}_i, \forall i \in Q\}) \wedge (|\overline{\mathcal{P}}_i| \leq d-1, \forall i \in Q) \wedge (|\overline{\mathcal{P}}_{\text{KG}}| \leq d-1)$$

Fig. 9: $(d-1)$ -probing IND-CCA for KEM security game

Confirming intuition, we prove that the $(d-1)$ -probing IND-CCA security notion gets stronger for larger allowed number of probes.

Lemma 16. *Let $d, t \in \mathbb{N}^*$ such that $t < d$. We have the following:*

$$(d-1)\text{-probing IND-CCA for KEM} \implies (t-1)\text{-probing IND-CCA for KEM}.$$

Proof. We observe that an adversary in the $(t-1)$ -probing IND-CCA security game already defines an adversary in the $(d-1)$ -probing IND-CCA security experiment, as all probes allowed in the former are allowed as well in the latter. Therefore, the advantage in the $(d-1)$ setting is lower-bounded by the advantage in the $(t-1)$ setting, by definition of the winning conditions. $\square$

In particular, since the 0 -probing IND-CCA game coincides with the standard IND-CCA game, we have the following corollary.

Corollary 2. *Let $d \in \mathbb{N}^*$ the number of probes allowed,*

$$(d-1)\text{-probing IND-CCA for KEM} \implies \text{IND-CCA for KEM}.$$

Remark 2. The proof would go the same for $(d-1)$ -probing IND-CPA/IND-CCA security for PKE schemes.

B.2 Transforming Probing IND-CCA PKE to KEM

We now prove that one can build a $(d-1)$ -probing IND-CCA secure KEM from a $(d-1)$ -probing IND-CCA secure PKE in the same manner as in the "no probing" case.

Theorem 3. *Let $(\text{KeyGen}, \text{Enc}, \text{Dec})$ be a PKE which is $(d-1)$ -probing IND-CCA secure and let $\mathcal{K}$ be its message space. It holds that $(\text{KeyGen}, \text{Encaps}, \text{Decaps})$ is a $(d-1)$ -probing IND-CCA KEM with key space $\mathcal{K}$, where Encaps and Decaps are defined as follows:*

Algorithm 9 Encaps	Algorithm 10 Decaps
Input: public key pk	Input: private key sk public key pk ciphertext ct
Output: ciphertext ct ephemeral key K	Output: ephemeral key K
$K \leftarrow \mathcal{K}$	$K := \text{Dec}(\text{pk}, \text{sk}, \text{ct})$
$\text{ct} \leftarrow \text{Enc}(\text{pk}, K)$	

Proof. We start by describing the reduction. Let $\mathcal{A}$ be an adversary for the $(d-1)$ -probing IND-CCA for KEM game. We are going to build an adversary $\mathcal{B}$ for the $(d-1)$ -probing IND-CCA for PKE game using $\mathcal{A}$. The adversary $\mathcal{B}$ receives a set of probes $\overline{\mathcal{P}}_{\text{KG}}$ on the key generation procedure for the KEM from the adversary $\mathcal{A}$. Then $\mathcal{B}$ forwards the exact same set of probes as probes on the $(d-1)$ -probing IND-CCA for PKE key generation procedure. Upon receiving a public key and a leakage $\mathcal{L}_{\text{KG}}$ from the key generation as input, $\mathcal{B}$ forwards them as they are to the adversary $\mathcal{A}$ as input for the $(d-1)$ -probing IND-CCA for KEM game. The adversary $\mathcal{B}$ then samples and outputs two messages K_0 and K_1 from $\mathcal{K}$ at random, and receives back the challenge ciphertext ct^* . It forwards

to the adversary $\mathcal{A}$ the challenge ciphertext together with the message K_0 as the $(d-1)$ -probing IND-CCA for KEM challenge. The adversary $\mathcal{B}$ then forwards all decryption with probes queries $(\text{ct}_i, \overline{\mathcal{P}}_i)$ made by $\mathcal{A}$ to the PKE decryption oracle, and forwards the decrypted plaintexts and leakages $(K_i, \mathcal{L}_i)$ back to $\mathcal{A}$. When the latter outputs a guess b^* , the adversary $\mathcal{B}$ outputs the same bit b^* as their own guess.

We now argue that the adversary $\mathcal{B}$ perfectly simulates a challenger for the $(d-1)$ -probing IND-CCA for KEM game. The KEM and PKE key generation procedures are the same, therefore all valid probes on the KEM key generation procedure are valid for the PKE key generation procedure, and the output and leakage of the PKE follow the same distribution than for the KEM key generation procedure. The challenge ciphertext sent to $\mathcal{A}$ by $\mathcal{B}$ is a PKE encryption of an ephemeral key sampled at random, which is exactly the definition of **Encaps**. The message K_0 is a valid challenge ephemeral key because the key space and message space are chosen to be identical in this transformation. As for the key generation procedure, the decryption and decapsulation procedures are exactly the same, therefore all inputs, outputs, probes and leakages on **Decaps** are perfectly simulated.

Finally, we argue that if $\mathcal{A}$ wins their game, then $\mathcal{B}$ also does. If the challenge encryption given as input to $\mathcal{B}$ is an encryption of K_0 , then the challenge ciphertext sent to $\mathcal{A}$ is an encryption of the challenge ephemeral key K_0 . When $\mathcal{A}$ wins, they thus output $b^* = 0$, which is also the correct guess for $\mathcal{B}$ to make. Also, $\mathcal{A}$ winning means that all conditions on the number of probes have been respected; since $\mathcal{B}$ has the same conditions on the number of probes and since they only sent the probes received from $\mathcal{A}$, $\mathcal{B}$ also meets the winning conditions on probes. Therefore, $\mathcal{B}$ also wins. On the other hand, when the challenge ciphertext given as input to $\mathcal{B}$ is an encryption not of K_0 but of K_1 , then the challenge ciphertext sent to $\mathcal{A}$ is a valid output of the encapsulation procedure but independent on K_0 . Thus $\mathcal{A}$ outputs $b^* = 1$ when winning, which is also the correct guess for $\mathcal{B}$ to make. For the same reasons than in the previous setting, winning conditions regarding probes also are met for $\mathcal{B}$ in this instance. Therefore, whenever $\mathcal{A}$ wins, $\mathcal{B}$ also does, and the reduction is complete. $\square$

B.3 Missing Proofs of Theorem 1

Proof of Correctness

Proof. Decryption as described in Algorithm 3 of an honestly generated ciphertext fails if either the NIZK proof is invalid or the decryption of the underlying $(d-1)$ -probing IND-CPA secure fails (under sk_1). The first case is upper bounded by $\epsilon_{\text{complete}}(\kappa)$, as the honest encryption guarantees $(x, w) \in R_\Sigma$. The second case is upper bounded by $\delta_{\text{fail}}(\kappa)$. A union bound gives the claimed correctness. $\blacksquare$

Proof of Lemma 3

Proof. For simplicity, we omit the dependency on the security parameter κ . We prove the following. Let $\mathcal{A}$ be a PPT adversary distinguishing between Hybrid_2 and Hybrid_3 with advantage at least ϵ_{ZK} , then we build a PPT adversary $\mathcal{B}$ that distinguishes between experiments $\text{Expt}_{\mathcal{A}}^{\text{ZK-Real}}$ and $\text{Expt}_{\mathcal{A},\text{Sim}}^{\text{ZK-Sim}}$ with advantage at least ϵ_{ZK} . Thus, $\mathcal{B}$ breaks the zero-knowledge property of the OTSS-NIZK, leading to a contraction.

We start by describing the reduction. Adversary $\mathcal{B}$ first receives on input from $\mathcal{A}$ a set of probes on the key generation procedure $\overline{\mathcal{P}}_{KG}$. Both in Hybrid_2 and Hybrid_3 , the key generation procedure consist of two independent calls on $\text{KeyGen}_{\text{CPA}}^{\mathcal{E}}$. Therefore we can split the set of probes $\overline{\mathcal{P}}_{KG}$ in two distinct sets: $\overline{\mathcal{P}}_{KG1}$ the probes on pk_1 generation and $\overline{\mathcal{P}}_{KG2}$ the probes on pk_2 generation. The adversary $\mathcal{B}$ computes the two IND-CPA public keys pk_1 and pk_2 as described both in Hybrid_2 and Hybrid_3 . Leakages for $\overline{\mathcal{P}}_{KG1}$ and $\overline{\mathcal{P}}_{KG2}$ are output by $\text{KeyGen}_{\text{CPA}}^{\mathcal{E}}$. We then forward the requested leakage $\mathcal{L}_{KG}$ and the public key $\text{pk} = (\text{pk}_1, \text{pk}_2)$ to $\mathcal{A}$. The adversary $\mathcal{B}$ computes all decryption queries as described both in Hybrid_2 and Hybrid_3 . After receiving m_0 and m_1 from $\mathcal{A}$, adversary $\mathcal{B}$ computes ct_1 (resp. ct_2) as an encryption of m_0 under pk_1 (resp. pk_2). It then sets $\mathbf{x} = (\text{pk}_1, \text{pk}_2, \text{ct}_1, \text{ct}_2)$ and $\mathbf{w} = (m, R_1, R_2)$ where $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\Sigma}$. Upon outputting $(\mathbf{x}, \mathbf{w})$, $\mathcal{B}$ receives a proof π in the zero-knowledge experiment for statement $\mathbf{x}$. The reduction $\mathcal{B}$ then forwards the challenge ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2, \pi)$ to $\mathcal{A}$ and computes all following decryption queries accordingly to description of Hybrid_2 (which is the same as in Hybrid_3). The reduction $\mathcal{B}$ outputs as their guess $\text{Expt}_{\mathcal{A}}^{\text{ZK-Real}}$ if $\mathcal{A}$ guessed Hybrid_2 , and $\text{Expt}_{\mathcal{A},\text{Sim}}^{\text{ZK-Sim}}$ if $\mathcal{A}$ guessed Hybrid_3 .

We have that the IND-CPA public keys, the leakage on the public key, the decryption queries and the IND-CPA encryptions of m_0 on public keys pk_1 and pk_2 are perfectly simulated as they were computed accordingly to the description of Hybrid_2 and Hybrid_3 , which coincide on these parts. When $\mathcal{B}$ plays the $\text{Expt}_{\mathcal{A}}^{\text{ZK-Real}}$ experiment, we have that $\mathcal{H}$ is a truly random oracle and that the proof π is computed using the prover $\mathcal{P}$, which is exactly the description of Hybrid_2 . When $\mathcal{B}$ plays the $\text{Expt}_{\mathcal{A},\text{Sim}}^{\text{ZK-Sim}}$ experiment, $\mathcal{H}$ and the proof are simulated, which is exactly the description of Hybrid_3 .

As in both situation $\mathcal{B}$ computes either hybrid exactly as they are described, it follows that whenever $\mathcal{A}$ wins, the adversary $\mathcal{B}$ then also wins. This concludes the reduction. $\blacksquare$

Proof of Lemma 4

Proof. For simplicity, we omit the dependency on the security parameter κ . Let $\mathcal{A}$ be a PPT adversary distinguishing between Hybrid_3 and Hybrid_4 with advantage at least $\text{Adv}_{\mathcal{A}}^{\text{dCPA}}$, then we build a PPT adversary $\mathcal{B}$ that breaks the $(d-1)$ -probing IND-CPA security of Σ with advantage at least $\text{Adv}_{\mathcal{A}}^{\text{dCPA}}$.

We begin by describing the reduction. First $\mathcal{B}$ receives from $\mathcal{A}$ a set of probes $\overline{\mathcal{P}}_{KG}$ on the key generation procedure. As seen in Lemma 3's proof, we can split the set of probes $\overline{\mathcal{P}}_{KG}$ in two distinct sets of probes: $\overline{\mathcal{P}}_{KG1}$ the probes on pk_1 generation and $\overline{\mathcal{P}}_{KG2}$ the probes on pk_2 generation. The adversary $\mathcal{B}$ outputs the

set of probes $\overline{\mathcal{P}}_{KG2}$ as their query and receives $(\mathbf{pk}_2, \mathcal{L}_{KG2})$, an IND-CPA public key and the leakage associated to $\overline{\mathcal{P}}_{KG2}$. Adversary $\mathcal{B}$ then computes the first IND-CPA key pair $(\mathbf{pk}_1, \mathbf{sk}_1)$, and the associated leakage $\mathcal{L}_{KG1}$ using $\text{KeyGen}_{\text{CPA}}^{\mathcal{L}}$. They simulate the random oracle $\mathbf{H}$ as described in both Hybrid_3 and Hybrid_4 . They then forward the public key $\mathbf{pk} = (\mathbf{pk}_1, \mathbf{pk}_2, r)$ and the requested leakage $\mathcal{L}_{KG} = (\mathcal{L}_{KG1}, \mathcal{L}_{KG2})$ to $\mathcal{A}$. When $\mathcal{B}$ receives m_0 and m_1 as input by $\mathcal{A}$, they forward the messages as output and get a $(d-1)$ -probing IND-CPA challenge ct_2 as input. They then compute ct_1 as an encryption under $\mathbf{pk}_1$ of m_0 , simulate a proof π for the statement $\mathbf{x} = (\mathbf{pk}_1, \mathbf{pk}_2, \text{ct}_1, \text{ct}_2)$ and output the challenge ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2, \pi)$ to $\mathcal{A}$. As they know the secret key $\mathbf{sk}_1$, $\mathcal{B}$ can answer all decryption queries by proceeding as described both in Hybrid_3 and Hybrid_4 . The reduction $\mathcal{B}$ outputs as their guess $b^* = 0$ if $\mathcal{A}$ guesses Hybrid_3 , and $b^* = 1$ if $\mathcal{A}$ guesses Hybrid_4 .

All decryption queries, the random oracle $\mathbf{H}$ and the challenge proof are perfectly simulated as they are computed exactly as described both in Hybrid_3 and Hybrid_4 . Both IND-CPA public keys are output by the same probabilistic function $\text{KeyGen}_{\text{CPA}}^{\mathcal{L}}$ even though only one of those is computed by $\mathcal{B}$ and the same goes for the challenge ciphertext IND-CPA encryptions. Let b be the secret input bit to the $(d-1)$ -probing IND-CPA game. When b equals 0, the challenge ciphertext output by $\mathcal{B}$ follows the exact same distribution than in Hybrid_3 , and when b is equal to 1, the challenge ciphertext follows the exact same distribution than in Hybrid_4 . Thus, we perfectly simulate the views of $\mathcal{A}$.

When $\mathcal{A}$ correctly guessed Hybrid_3 , this means that the challenge ciphertext $\mathcal{B}$ sent was twice an encryption of m_0 . Thus, $b^* = 0$ was a correct guess for $\mathcal{B}$. If $\mathcal{A}$ correctly guessed Hybrid_4 , this means that the challenge ciphertext's second IND-CPA encryption ct_2 was an encryption of m_1 . Thus, $b^* = 1$ was a correct guess for $\mathcal{B}$. Therefore, whenever $\mathcal{A}$ wins, $\mathcal{B}$ also does, which completes the reduction. $\blacksquare$

Appendix C Missing Details of Section 4

C.1 Proof of Correctness of Rutile

We provide the full proof of Lemma 8.

Proof. As masking gadgets do not impact correctness, we can directly look at the equations in the decoded version. It holds

$$\begin{aligned} \mu' &= v - \mathbf{v}^t \cdot \mathbf{s} \\ &= \mathbf{u}^t \cdot (\mathbf{A}\mathbf{s} + \mathbf{e}) + e_2 + \lfloor q/2 \rfloor \mu - (\mathbf{u}^t \mathbf{A} + \mathbf{e}_1^t) \mathbf{s} \\ &= \lfloor q/2 \rfloor \mu + \mathbf{u}^t \mathbf{e} - \mathbf{e}_1^t \mathbf{s} + e_2. \end{aligned}$$

Hence, decryption succeeds if $\|\mathbf{u}^t \mathbf{e} - \mathbf{e}_1^t \mathbf{s} + e_2\|_{\infty} \leq \lfloor q/4 \rfloor$. We set $\mathbf{r} = (\mathbf{u}, \mathbf{e}_1, e_2)$ which is sampled from $\text{SUnif}(\mathcal{R}, 2k+1, u, d \cdot \text{rep})$ and set $\mathbf{z} = (\mathbf{e}, -\mathbf{s}, 1)$, with $(\mathbf{e}, \mathbf{s})$ sampled from $\text{SUnif}(\mathcal{R}, 2k, u, d \cdot \text{rep})$. By Equation 1 with probability at most $\exp(-2n(2k+1))$ it holds $\|\mathbf{r}\|_2 \leq B := 2^u \sqrt{3d \cdot \text{rep} \cdot n(2k+1)}$. Similarly,

we can upper bound $\|\mathbf{z}\|_2 \leq B$. Hence, by Lemma 1 and a union bound, it holds with probability at most $2 \exp(-2n(2k+1))$

$$\begin{aligned} \|\mathbf{u}^t \mathbf{e} - \mathbf{e}_1^t \mathbf{s} + e_2\|_\infty &= \|\mathbf{r}^t \mathbf{z}\|_\infty \\ &\leq \|\mathbf{r}\|_2 \cdot \|\mathbf{z}\|_2 \\ &\leq B^2 \\ &= 2^{2u} \cdot d \cdot \text{rep} \cdot n(6k+3). \end{aligned}$$

□

C.2 Proof of NY-Compatibility of Rutile

We provide the full proof of Lemma 9.

Proof. We set $\delta = \exp(-2n(2k+1))$ and let P_1 and P_2 as defined in Definition 17. First, we show that RandSpace is chosen in a way that honestly generated encryption randomness lies in RandSpace with overwhelming probability. By Eq. 1 applied to $\text{RandDistr} = \text{SUnif}(\mathcal{R}, 2k+1, u, d \cdot \text{rep})$, we obtain

$$P_1 = \Pr_{R \leftarrow \text{RandDistr}} [R \notin \text{RandSpace}] \leq \delta.$$

Second, we show that

$$\Pr_{(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}_{\text{CPA}}} [\exists (\mu, \mathbf{r}) \in \mathcal{M} \times \text{RandSpace} \mid \text{Dec}_{\text{CPA}}(\text{sk}, \text{Enc}_{\text{CPA}}(\text{pk}, \mu; \mathbf{r})) \neq \mu] \leq \delta,$$

which implies that $P_2 \leq 2\delta$. From the proof of Lemma 8, we know that decryption succeeds if $\|\mathbf{r}^t \cdot \mathbf{z}\|_\infty \leq \lfloor q/4 \rfloor$, where $\mathbf{z} = (\mathbf{e}, -\mathbf{s}, 1)$ comes from the key generation. In contrast to the correctness proof, we do not know whether $\mathbf{r}$ has been sampled correctly from RandDistr , but we know that $\mathbf{r} \in \text{RandSpace}$, hence $\|\mathbf{r}\|_2 \leq B^{(e)}$. As the key generation is assumed to be honest, we can further bound $\|(\mathbf{e}, -\mathbf{s}, 1)\|_2$ by $B^{(e)}$, except with probability $\delta = \exp(-2n(2k+1))$. As $\hat{\delta}_{\text{fail}} = \max(P_1, P_2) = 2\delta$, this concludes the proof. □

Appendix D Missing Details of Section 7

D.1 Proof of Lemma 13

Proof. We note $\mathcal{U}$ [resp. $\mathcal{U}'$] the $d \cdot 2\text{rep}$ small uniforms input to $\text{KeyGen}_{\text{ER}}^{\mathcal{L}}$ [resp. $\text{KeyGen}_{\text{ER}}$] and $(\text{pk}, \mathcal{L}_{KG})$ [resp. $(\text{pk}', \mathcal{L}'_{KG})$] the answer to $\mathcal{A}$'s first query in Hybrid_2 [resp. Hybrid_3]. Let $(\mathbf{A}, \mathbf{t}) \in \mathcal{R}_q^{k \times k} \times \mathcal{R}_q^k$ be a possible public key pair and l a possible leakage value for $\mathcal{L}_{KG}$. We argue that $\Pr[\text{pk} = (\mathbf{A}, \mathbf{t}), \mathcal{L}_{KG} = l] = \Pr[\text{pk}' = (\mathbf{A}, \mathbf{t}), \mathcal{L}'_{KG} = l]$, where the probability is defined over the random coins of SimLeak , $\text{KeyGen}_{\text{ER}}$ and the sampling of all $d \cdot 2\text{rep}$ small uniforms (ρ_i) .

We note $S := \{(\rho_i) \leftarrow \text{Unif}(\mathcal{R}, k, u)^{d \cdot 2\text{rep}} \mid \Pr[\mathcal{U} = (\rho_i), \text{pk} = (\mathbf{A}, \mathbf{t})] \neq 0\}$, the set of all values (ρ_i) compatible with $(\mathbf{A}, \mathbf{t})$. We have that contents of the set

do not change when considering $\mathbf{pk}'$ and $\mathcal{U}'$ instead of $\mathbf{pk}$ and $\mathcal{U}$, by similarities between the two hybrids.

We consider $\Pr[\mathbf{pk} = (\mathbf{A}, \mathbf{t}), \mathcal{L}_{KG} = l]$. By first applying Bayes formula on " $\mathbf{pk} = (\mathbf{A}, \mathbf{t})$ " and then applying law of total probability for all values in S for $\mathcal{U}$ we have the following:

$$\begin{aligned} & \Pr[\mathbf{pk} = (\mathbf{A}, \mathbf{t}), \mathcal{L}_{KG} = l] \\ &= \Pr[\mathbf{pk} = (\mathbf{A}, \mathbf{t})] \cdot \Pr[\mathcal{L}_{KG} = l | \mathbf{pk} = (\mathbf{A}, \mathbf{t})] \\ &= \Pr[\mathbf{pk} = (\mathbf{A}, \mathbf{t})] \cdot \\ & \quad \sum_{(\rho_i) \in S} \Pr[\mathcal{U} = (\rho_i) | \mathbf{pk} = (\mathbf{A}, \mathbf{t})] \cdot \Pr[\mathcal{L}_{KG} = l | \mathbf{pk} = (\mathbf{A}, \mathbf{t}), \mathcal{U} = (\rho_i)] \end{aligned}$$

Applying the same arguments and reasoning on $\Pr[\mathbf{pk}' = (\mathbf{A}, \mathbf{t}), \mathcal{L}'_{KG} = l]$ yields:

$$\begin{aligned} & \Pr[\mathbf{pk}' = (\mathbf{A}, \mathbf{t}), \mathcal{L}'_{KG} = l] \\ &= \Pr[\mathbf{pk}' = (\mathbf{A}, \mathbf{t})] \cdot \\ & \quad \sum_{(\rho_i) \in S} \Pr[\mathcal{U}' = (\rho_i) | \mathbf{pk}' = (\mathbf{A}, \mathbf{t})] \cdot \Pr[\mathcal{L}'_{KG} = l | \mathbf{pk}' = (\mathbf{A}, \mathbf{t}), \mathcal{U}' = (\rho_i)] \end{aligned}$$

By application of the Nluo property for the following first equality and by similarities between the two hybrids for the latter, we have the following:

$$\begin{aligned} \Pr[\mathcal{L}_{KG} = l | \mathbf{pk} = (\mathbf{A}, \mathbf{t}), \mathcal{U} = (\rho_i)] &= \Pr[\mathcal{L}'_{KG} = l | \mathbf{pk}' = (\mathbf{A}, \mathbf{t}), \mathcal{U}' = (\rho_i)] \\ \Pr[\mathbf{pk} = (\mathbf{A}, \mathbf{t})] &= \Pr[\mathbf{pk}' = (\mathbf{A}, \mathbf{t})] \\ \Pr[\mathcal{U} = (\rho_i) | \mathbf{pk} = (\mathbf{A}, \mathbf{t})] &= \Pr[\mathcal{U}' = (\rho_i) | \mathbf{pk}' = (\mathbf{A}, \mathbf{t})] \end{aligned}$$

Thus, $\Pr[\mathbf{pk} = (\mathbf{A}, \mathbf{t}), \mathcal{L}_{KG} = l] = \Pr[\mathbf{pk}' = (\mathbf{A}, \mathbf{t}), \mathcal{L}'_{KG} = l]$ and the views of $\mathcal{A}$ are the same in both hybrids, which concludes the proof. $\square$